Database Development and Class Registration

Implementation Evidence Supplement

Louis Murphy | CST499 | Charmelia Butler | October 28, 2026

Read this supplement alongside the four-page paper. Figures 1-34 document actual database structures, before-and-after operations, student pages, and PHP source. The project archive contains the original PNGs and the corresponding implementation.

Verification used PHP 8.3.6, MySQL 8.0.46, and phpMyAdmin 5.2.1 on Linux. All student and course records are fictional. Captures show the running backend and HTTP responses; they do not assert installation on a particular student computer.

Evidence index

Figures 1-13 Schema migration, all nine table structures, foreign keys, and seeded offerings.

Figures 14-24 Catalog, semester schedule, add and remove operations, and empty states.

Figures 25-30 Waiting list, reservation, offer acceptance, and profile editing.

Figures 31-34 Actual PHP source for transactions, actions, and schedule queries.

Before and after comparisons

Create tables: Figure 1 to Figure 2. First registration: Figure 16 to Figure 17.

Add another course: Figure 17 to Figure 19. Delete a class: Figure 19 to Figure 22.

Accept a reserved seat: Figure 28 to Figure 29.

Figure 1 Database Before Migration

Before migration: the Week 3 database contains accounts and profiles.

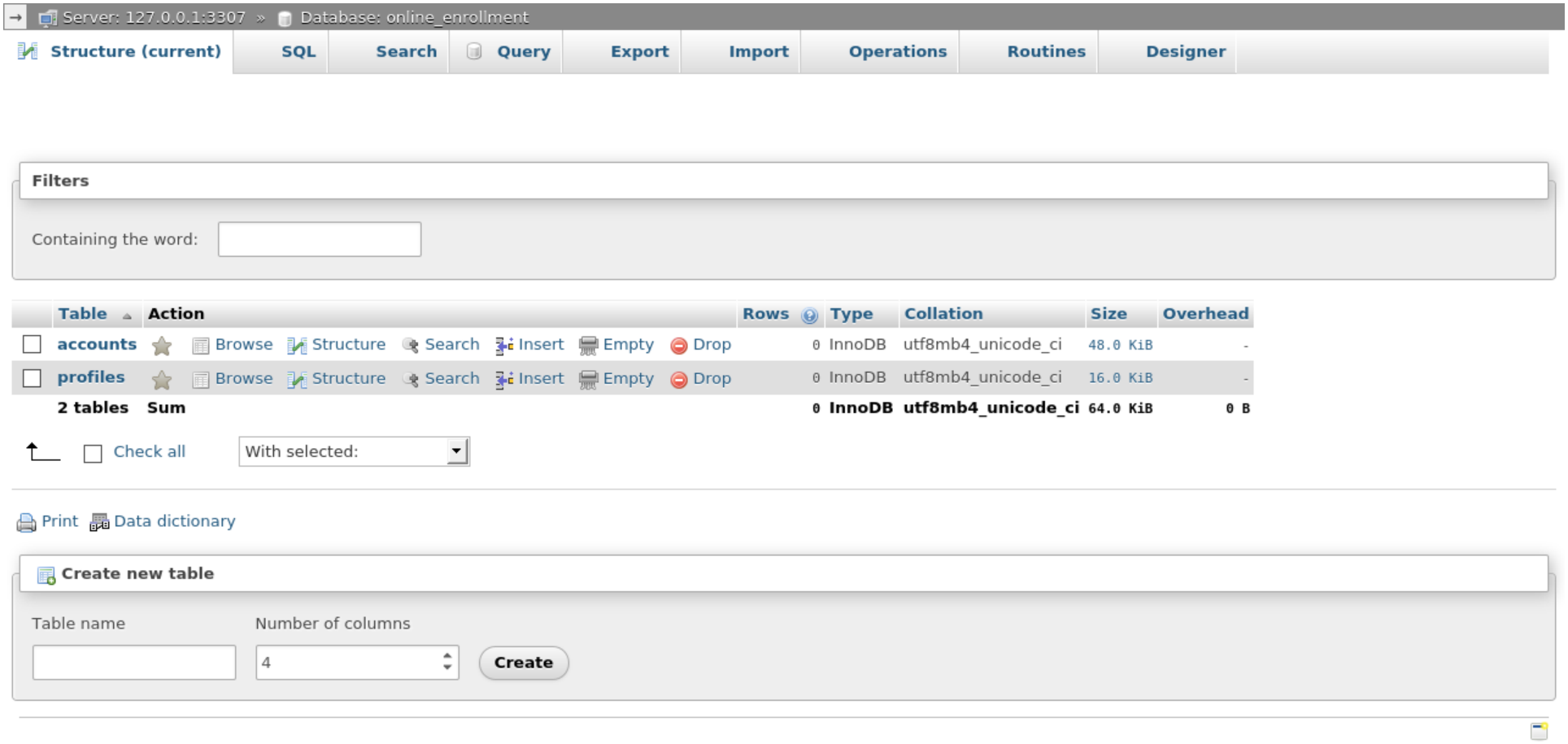


Figure 2 Database After Migration

After migration: nine InnoDB tables support accounts, semesters, enrollment, queues, and offers.

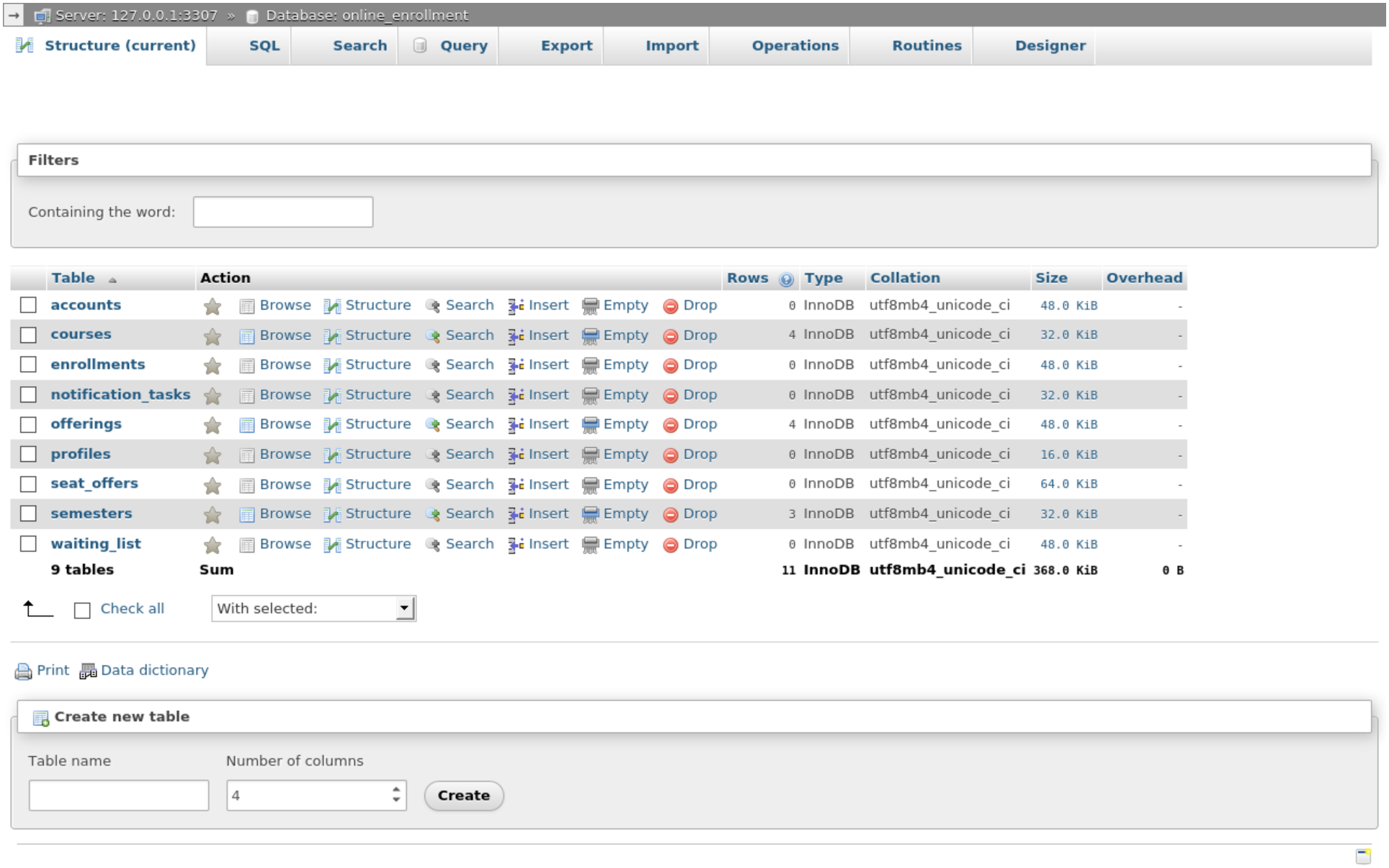


Figure 3 Accounts Table Structure

Actual phpMyAdmin Structure view of accounts: column names, types, nullability, defaults, and keys.

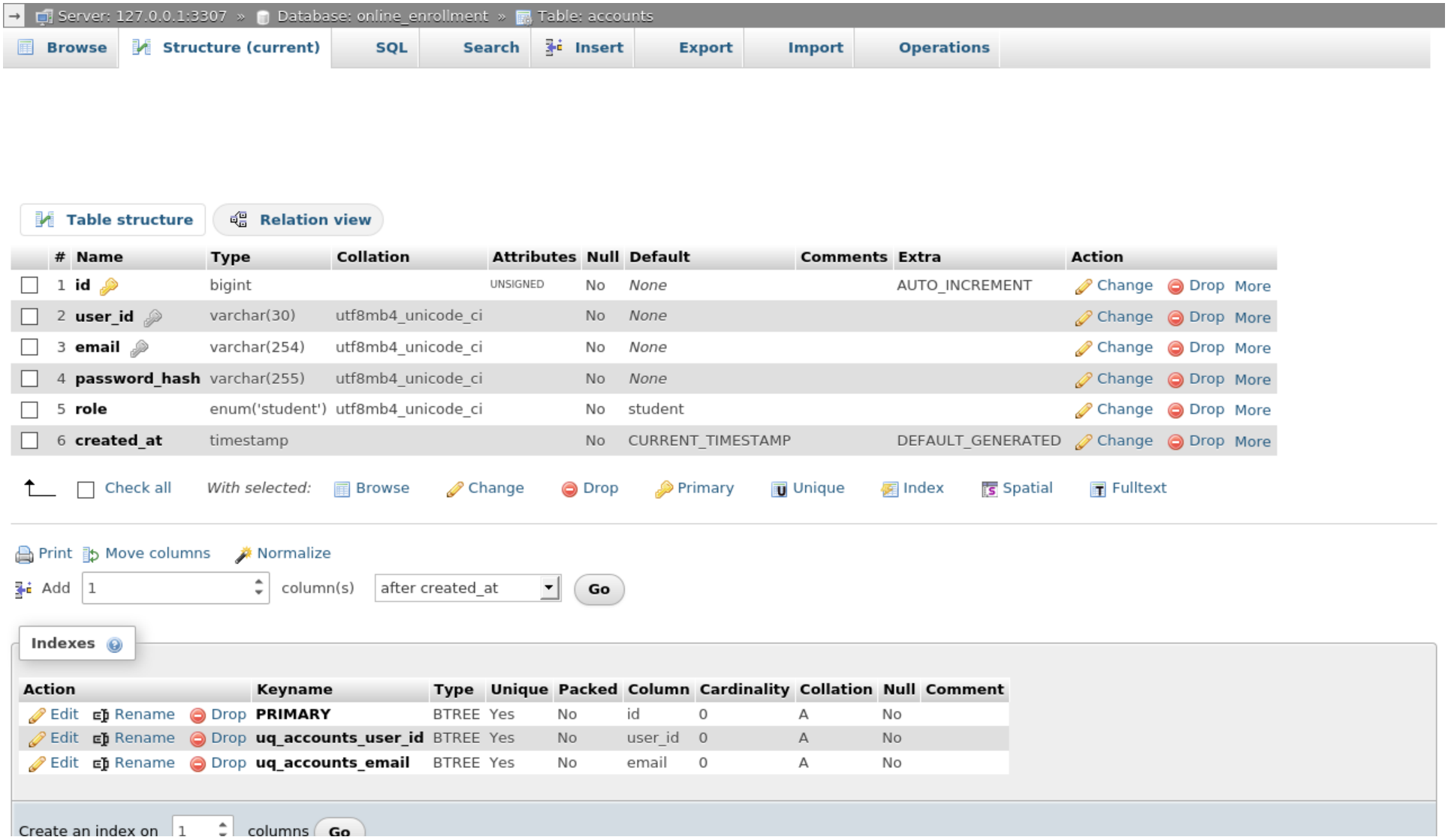


Figure 4 Profiles Table Structure

Actual phpMyAdmin Structure view of profiles: column names, types, nullability, defaults, and keys.

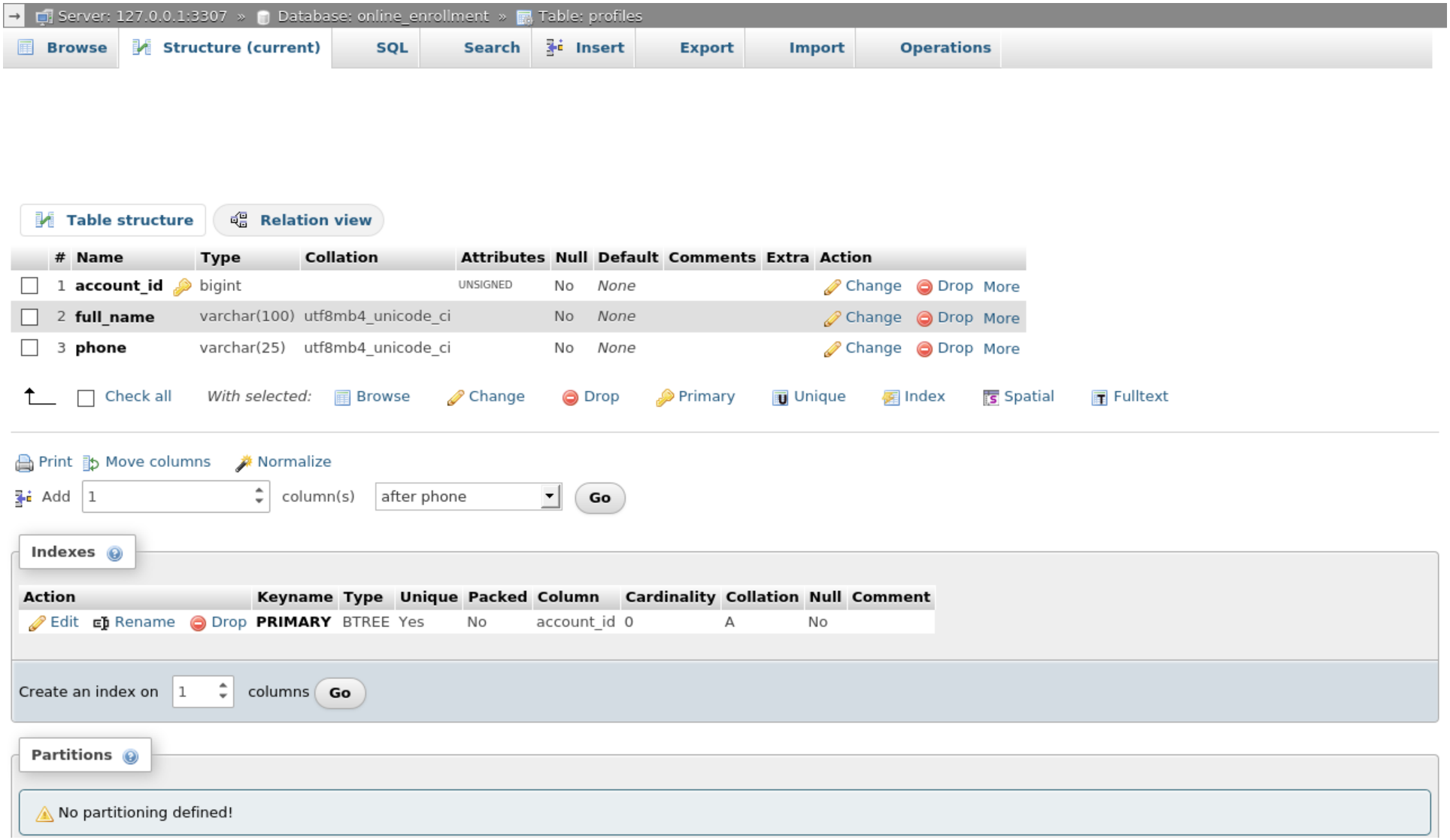


Figure 5 Courses Table Structure

Actual phpMyAdmin Structure view of courses: column names, types, nullability, defaults, and keys.

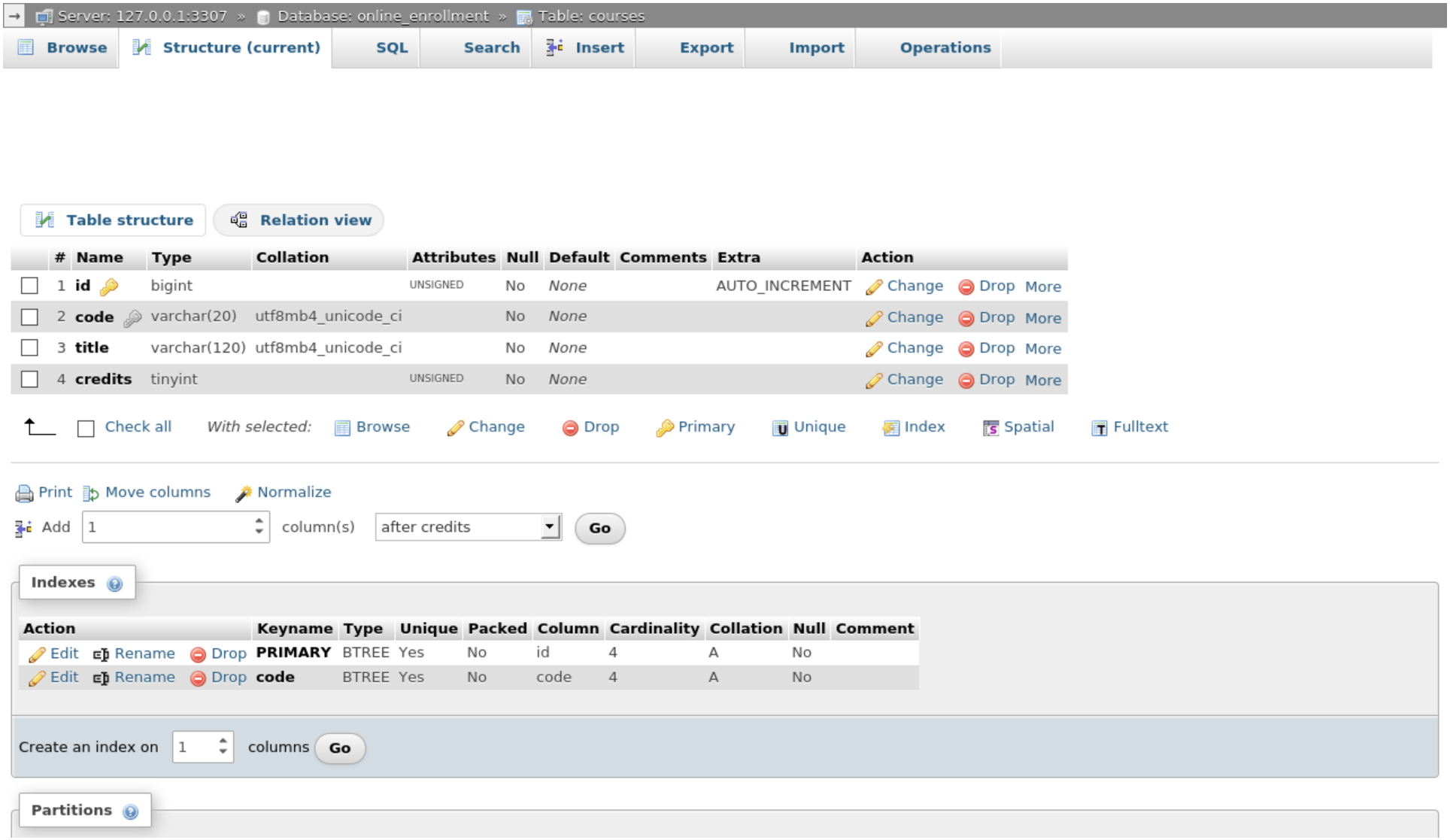


Figure 6 Semesters Table Structure

Actual phpMyAdmin Structure view of semesters: column names, types, nullability, defaults, and keys.

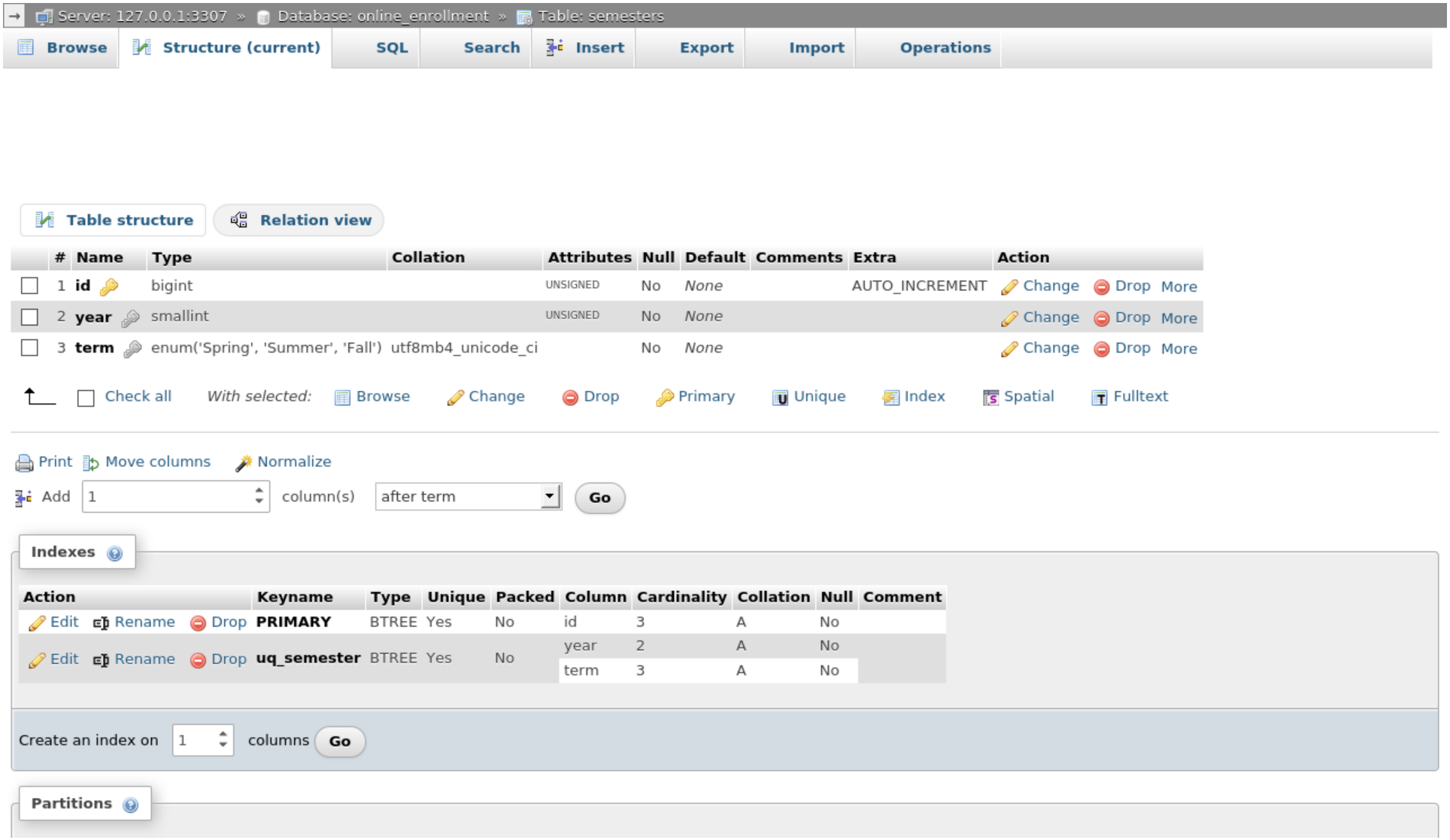


Figure 7 Offerings Table Structure

Actual phpMyAdmin Structure view of offerings: column names, types, nullability, defaults, and keys.

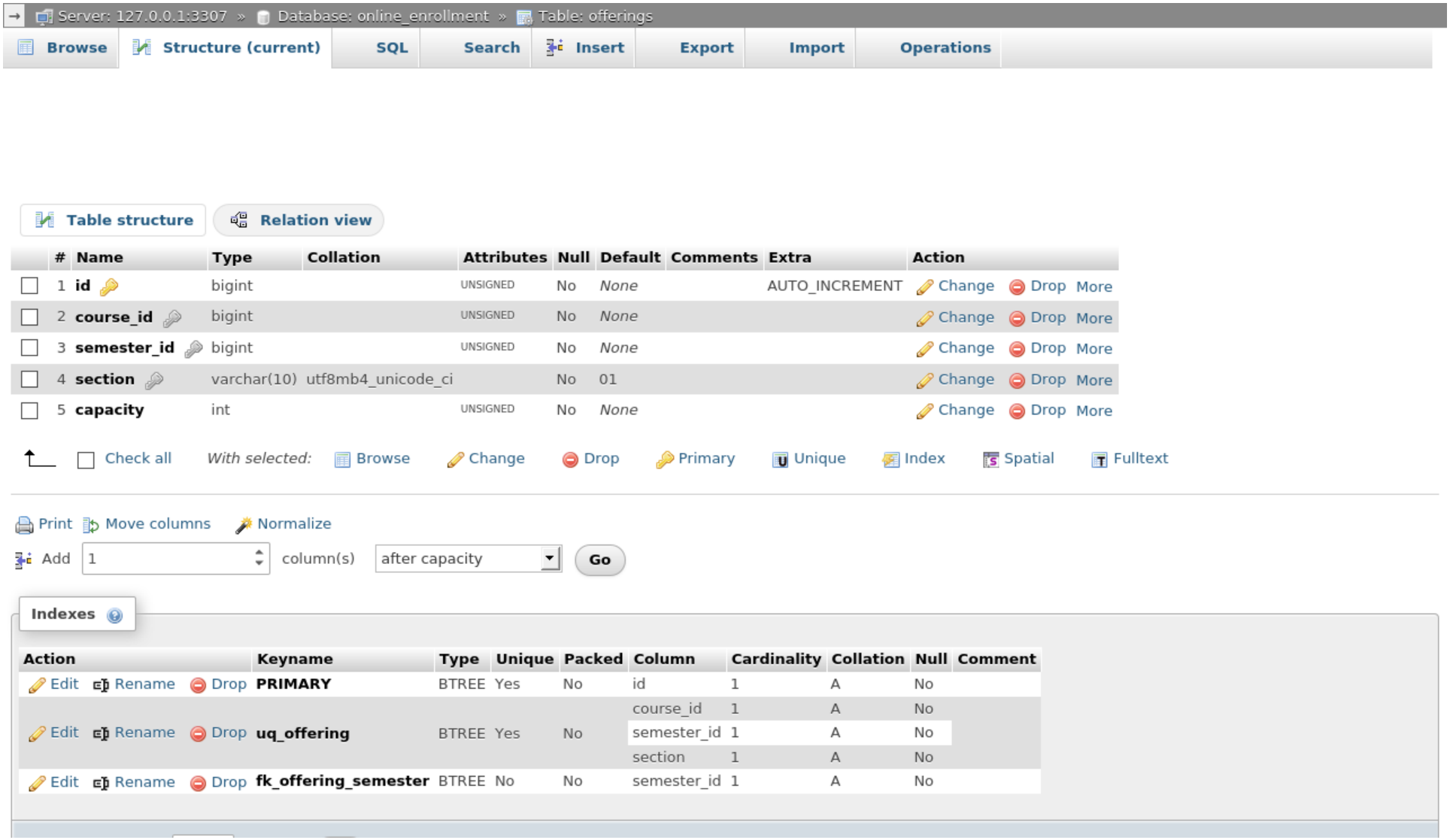


Figure 8 Enrollments Table Structure

Actual phpMyAdmin Structure view of enrollments: column names, types, nullability, defaults, and keys.

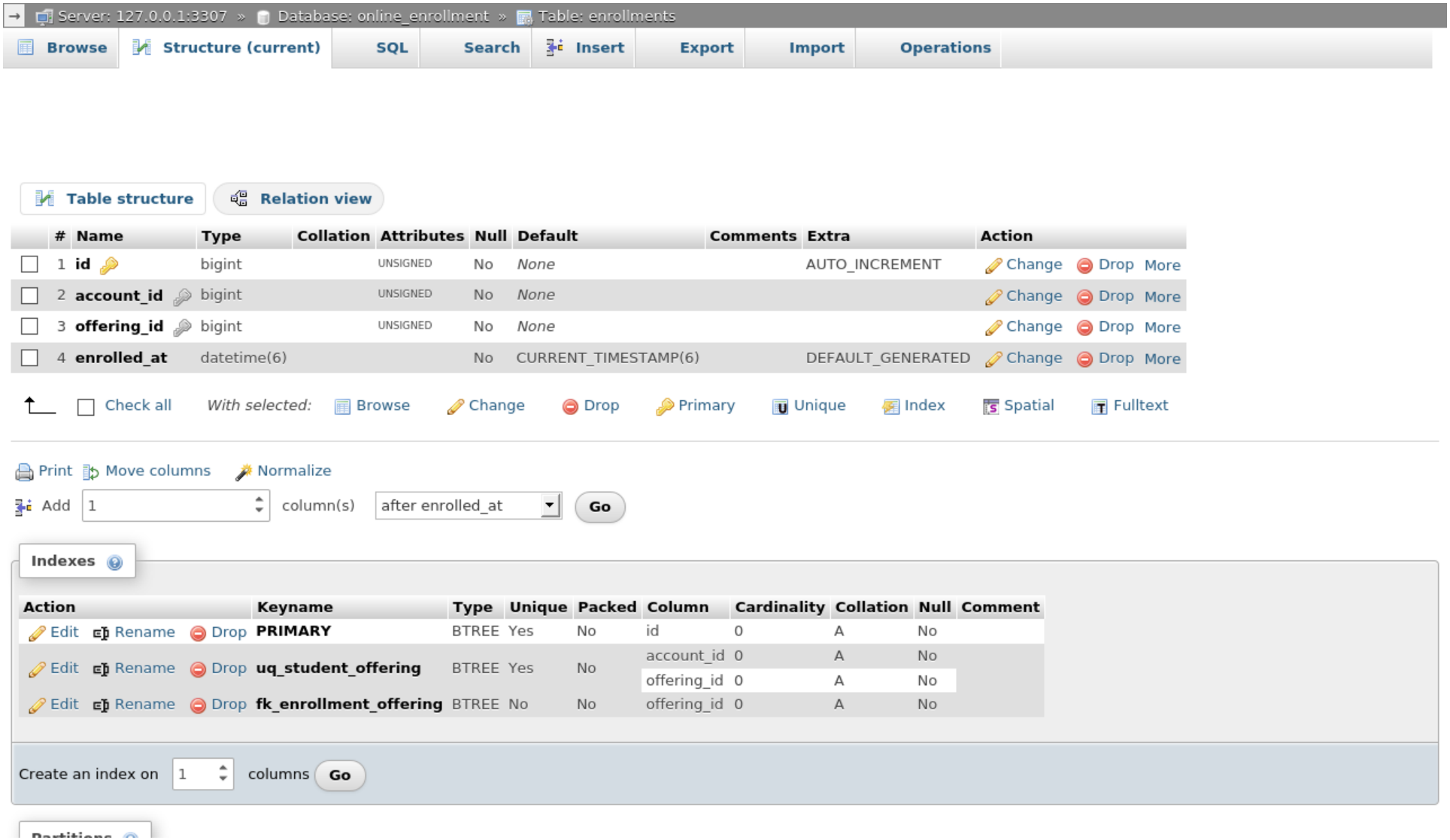


Figure 9 Waiting List Table Structure

Actual phpMyAdmin Structure view of waiting_list: column names, types, nullability, defaults, and keys.

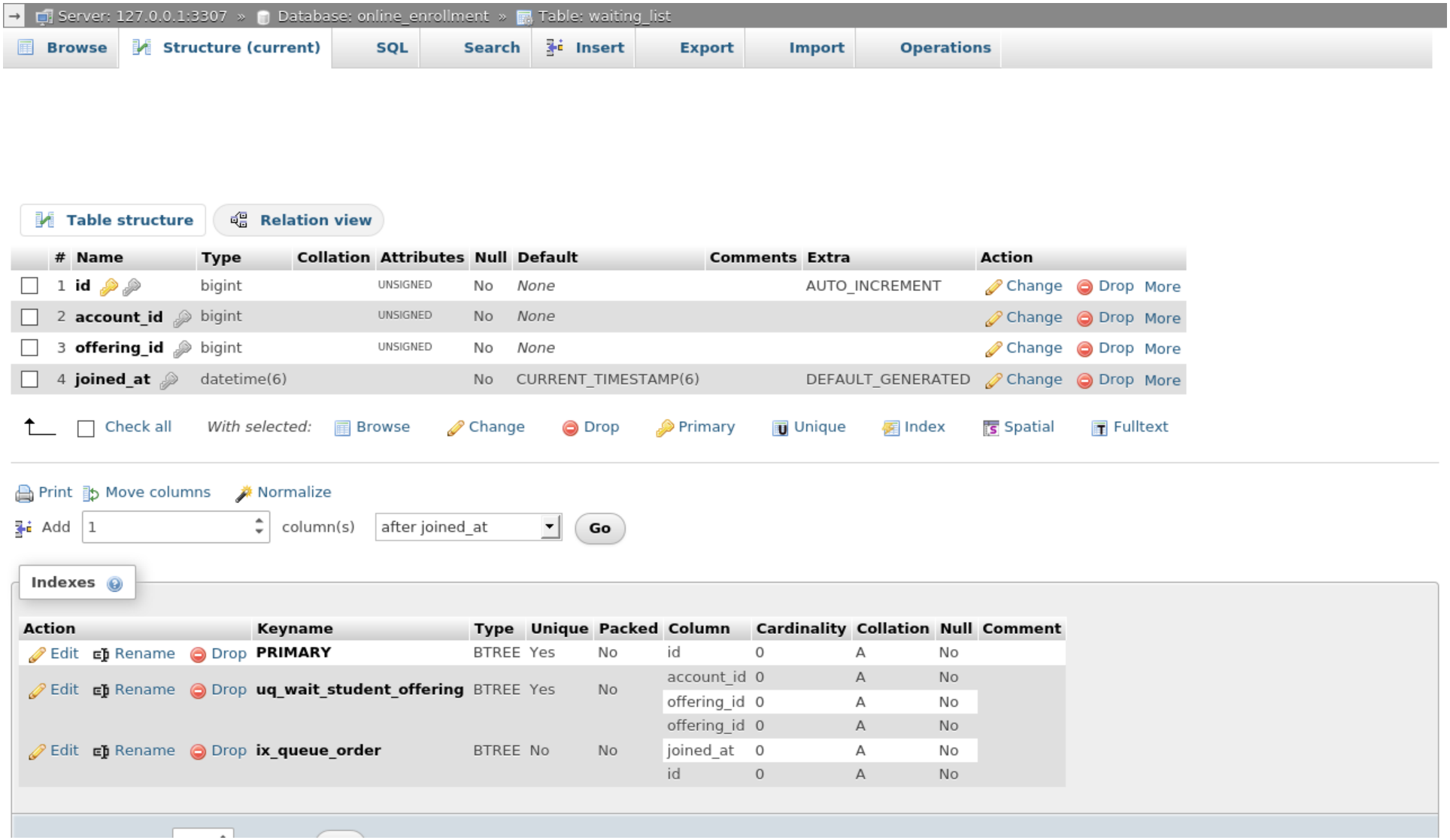


Figure 10 Seat Offers Table Structure

Actual phpMyAdmin Structure view of seat_offers: column names, types, nullability, defaults, and keys.

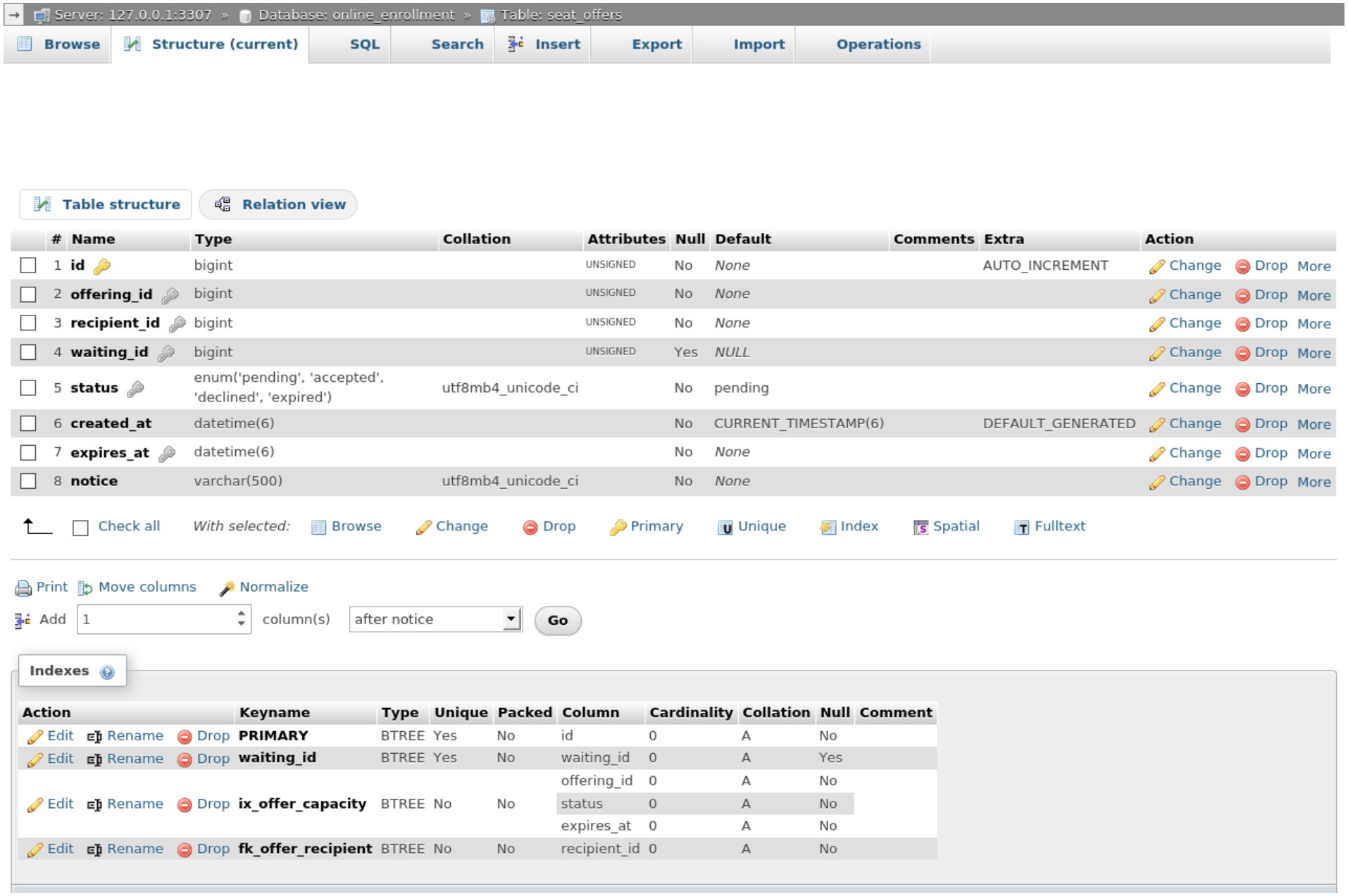


Figure 11 Notification Tasks Table Structure

Actual phpMyAdmin Structure view of notification_tasks: column names, types, nullability, defaults, and keys.

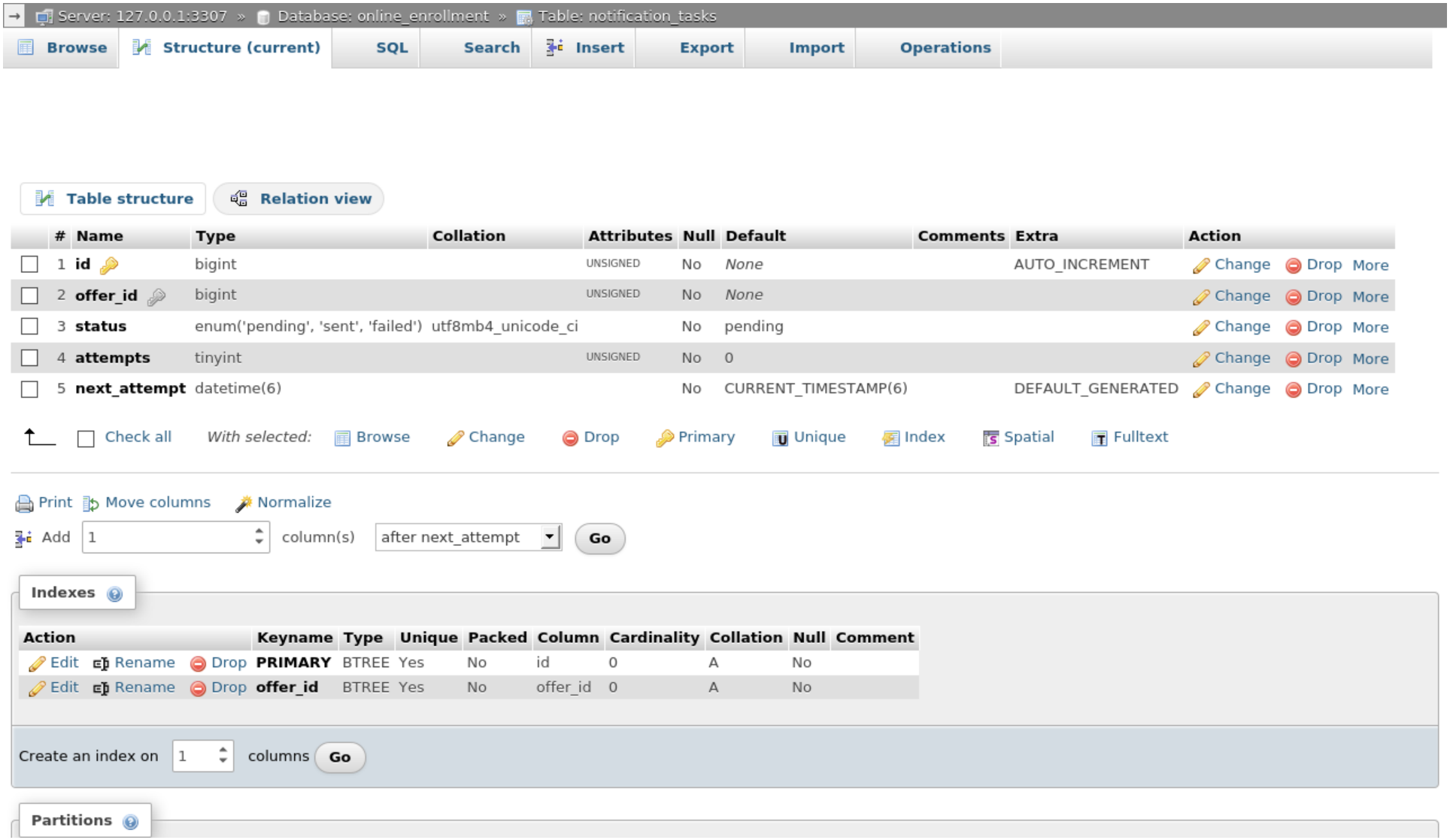


Figure 12 Enrollment Foreign Keys

Foreign-key relationships link enrollments.account_id to accounts.id and enrollments.offering_id to offerings.id.

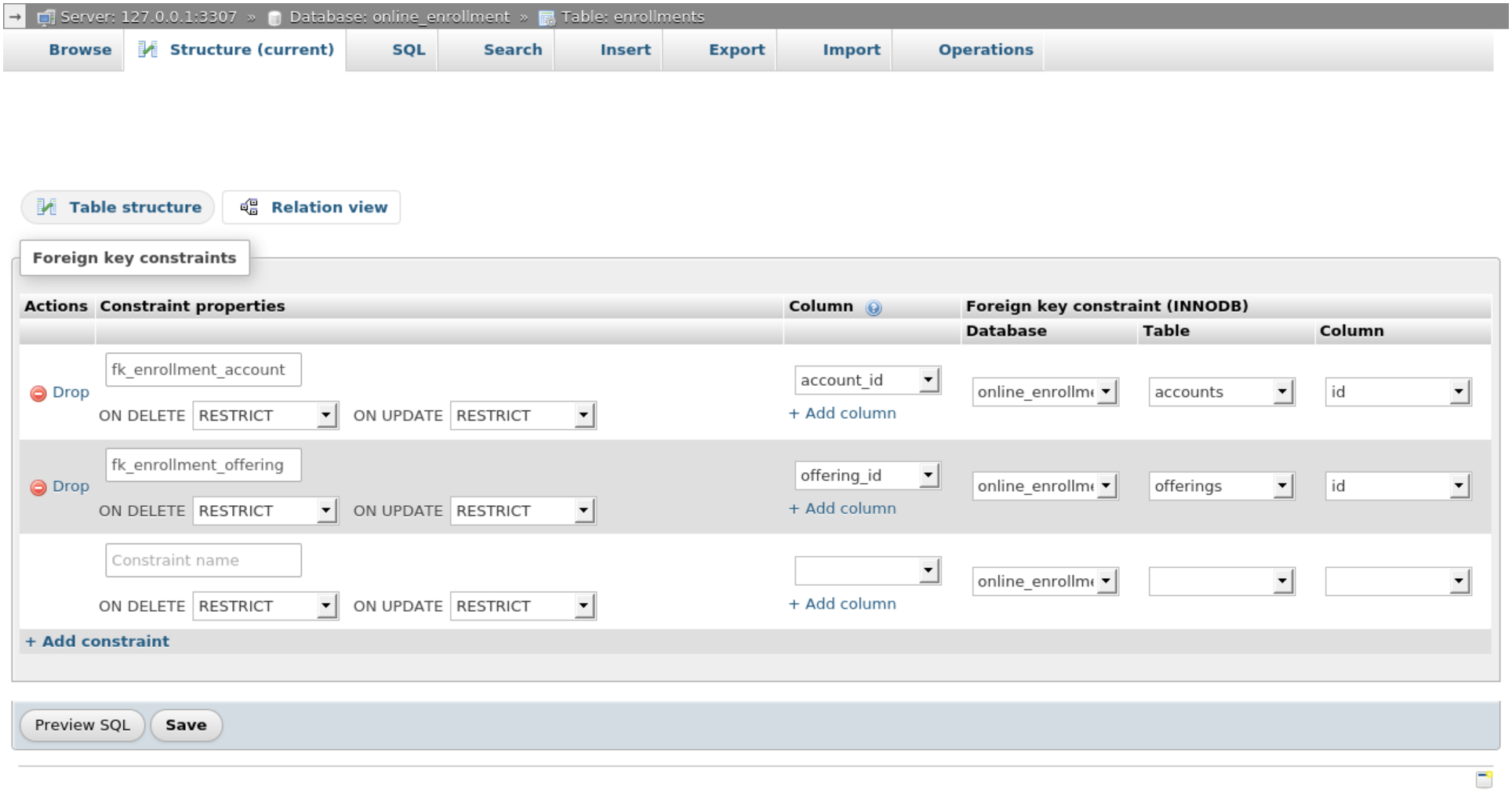


Figure 13 Seeded Offering Records

Seeded offerings identify courses, semesters, sections, and capacities.

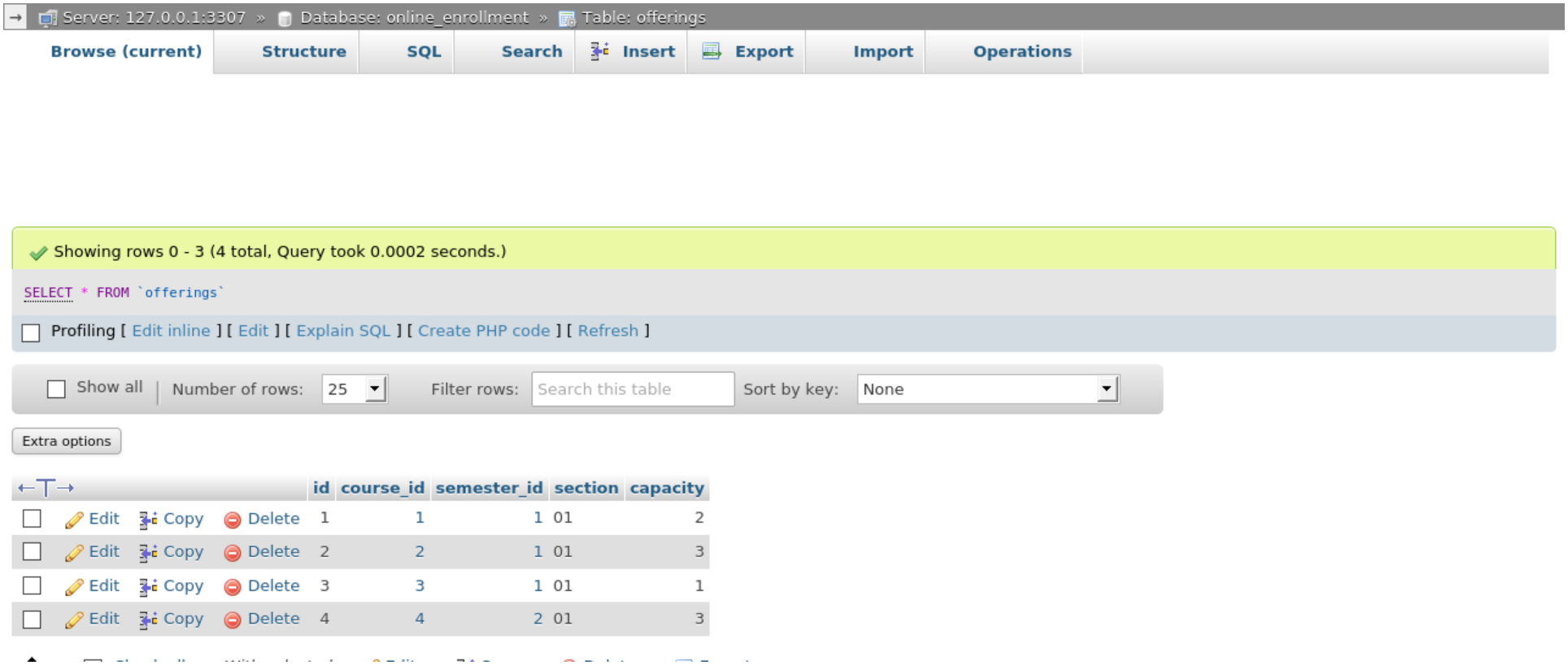


Figure 14 Course Catalog

Semester-filtered course catalog with capacities and registration actions.

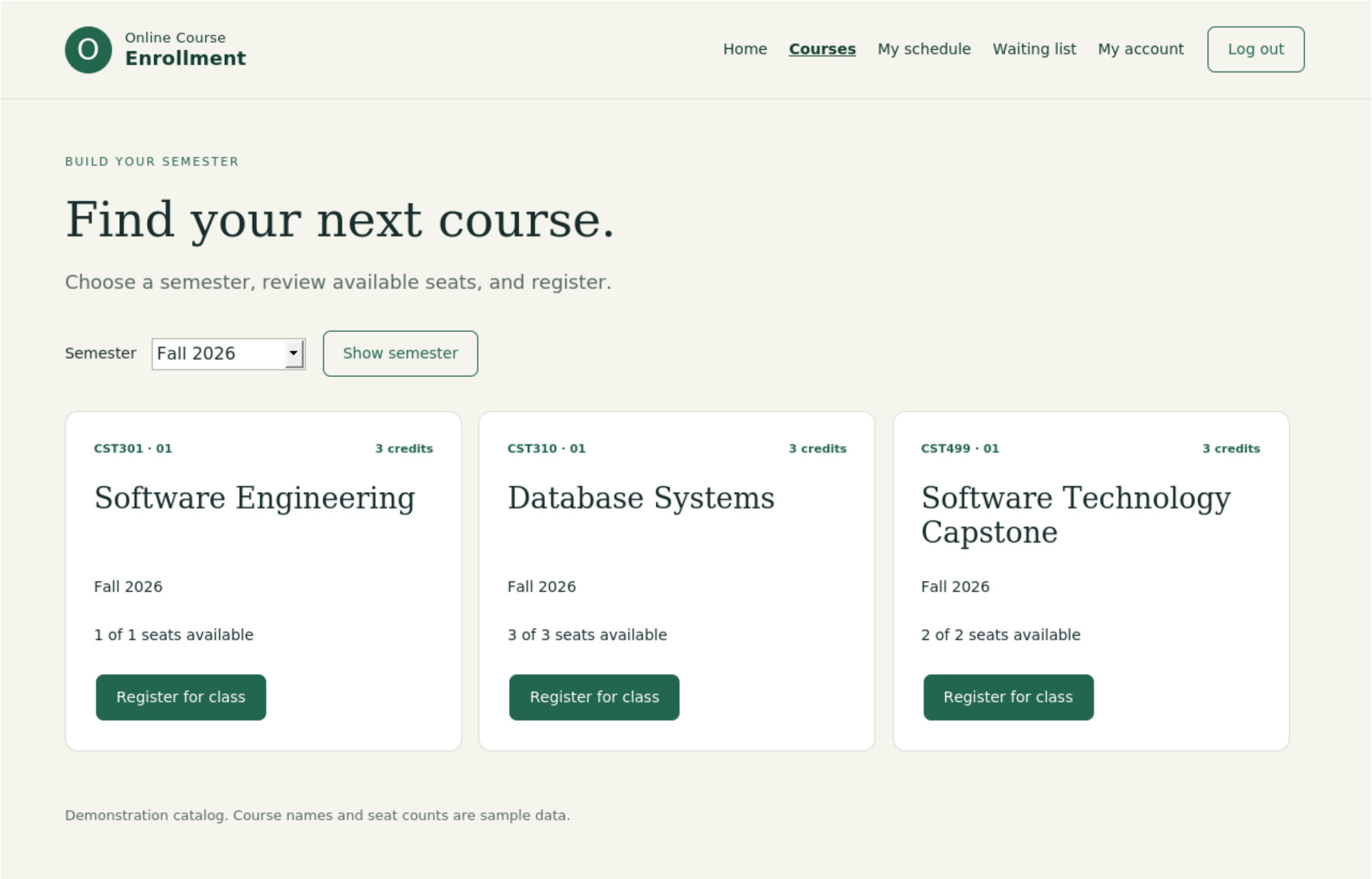


Figure 15 Empty Student Schedule

The student starts with an empty Fall 2026 schedule.

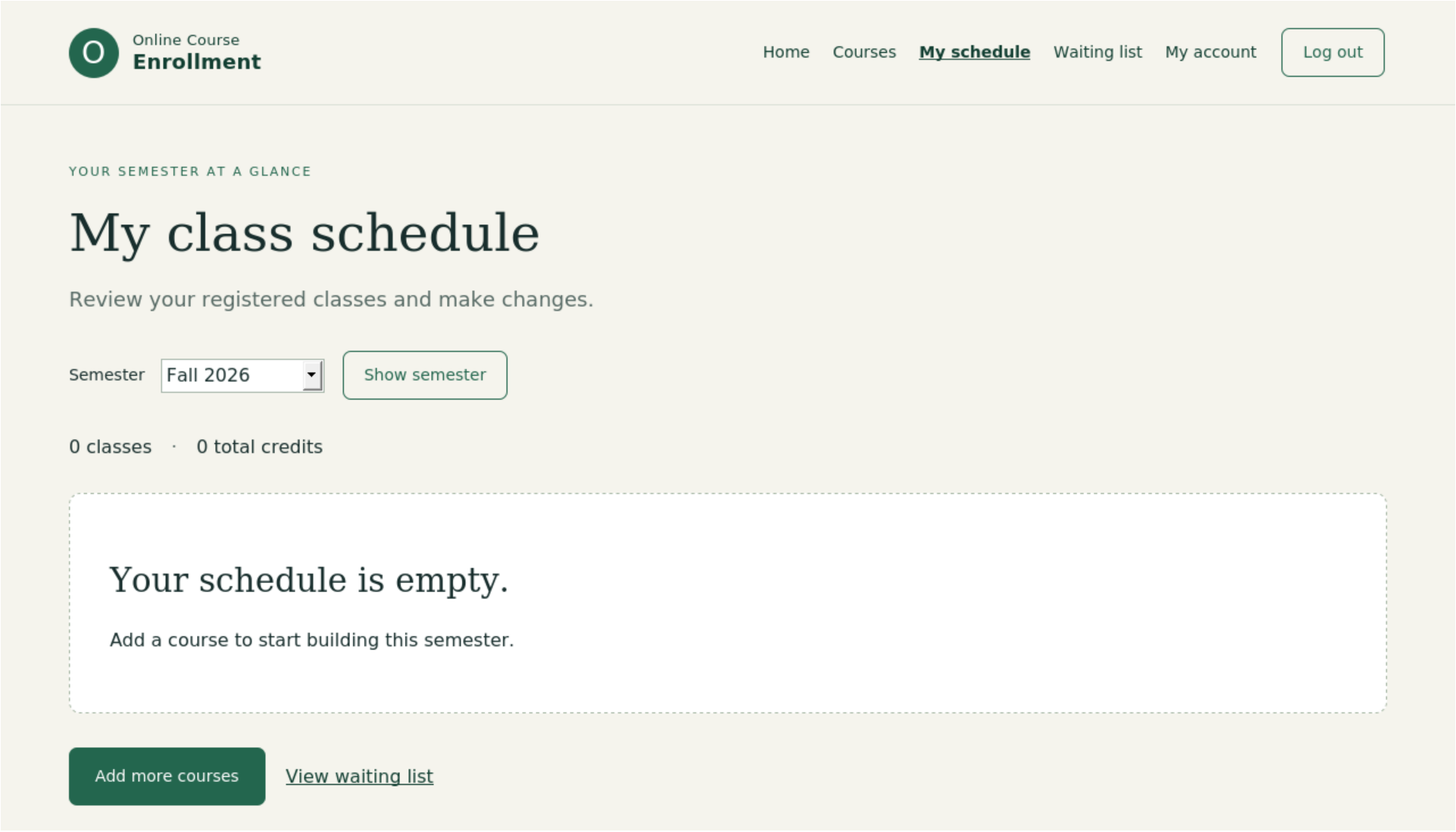


Figure 16 Before First Registration

Before first registration: enrollments contains zero rows.

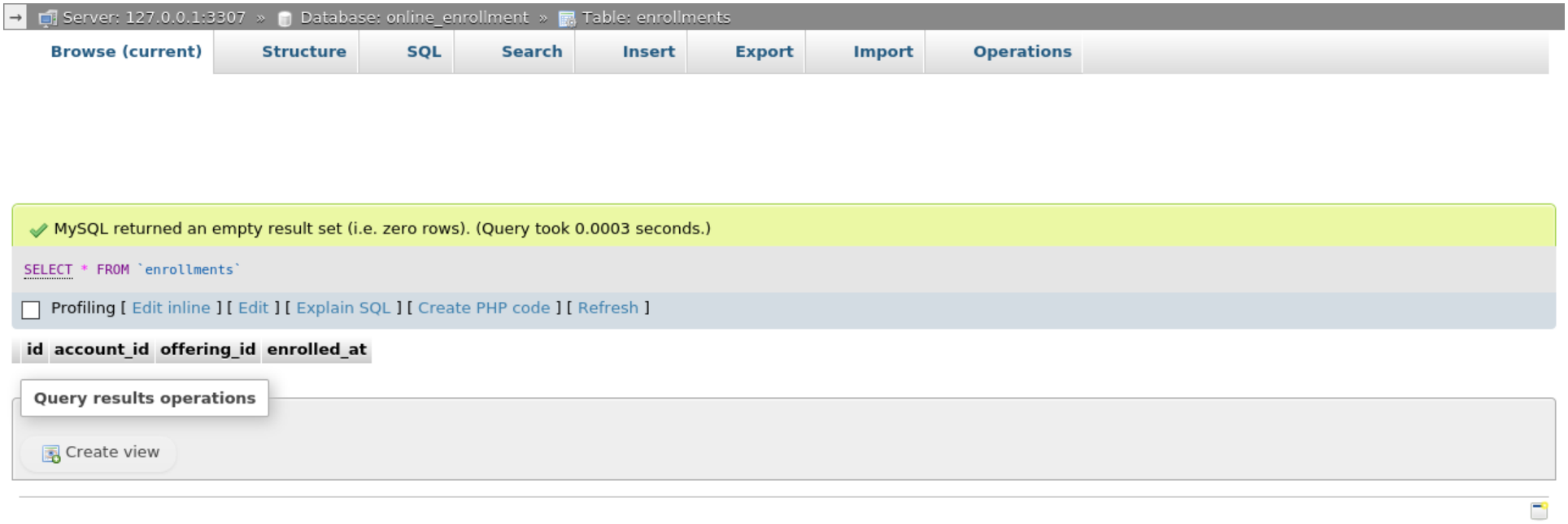


Figure 17 After First Registration

After first registration and before adding another class: Alex (account_id 1) is enrolled in offering 1.

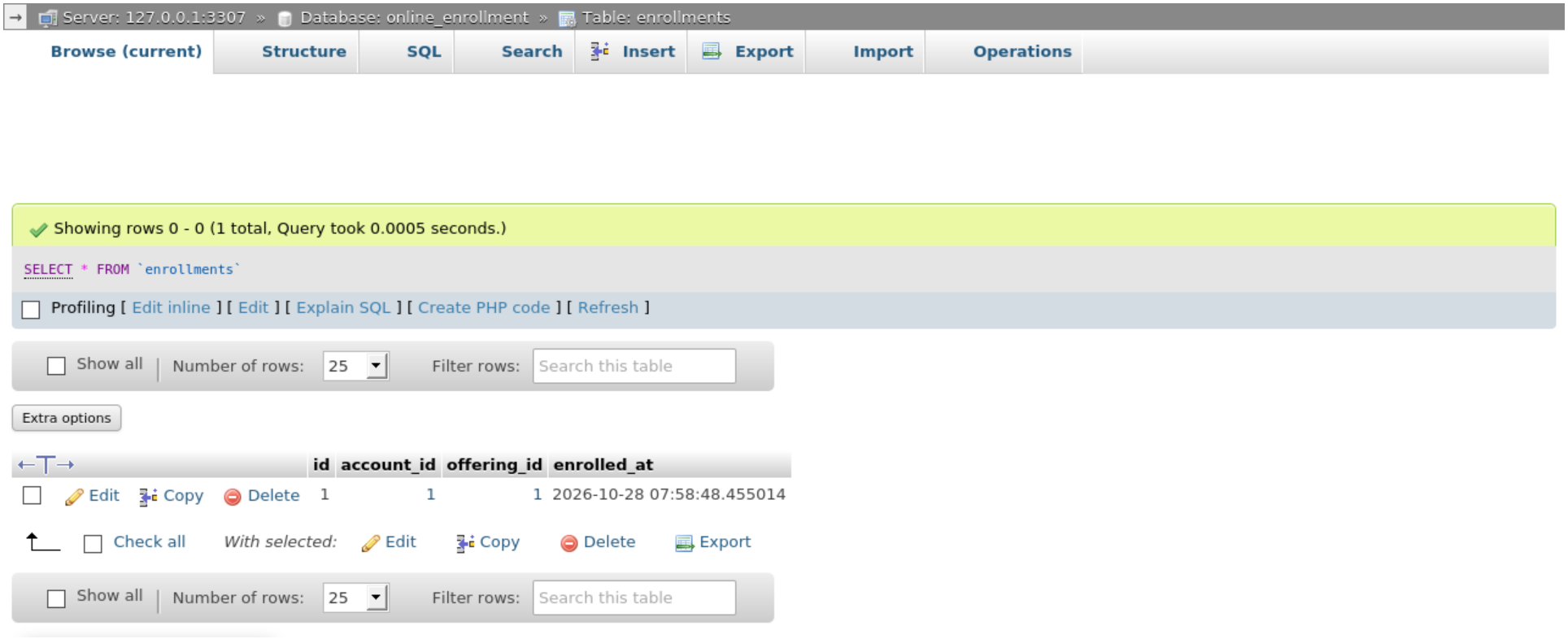


Figure 18 Schedule With One Class

The schedule displays the first registered class.

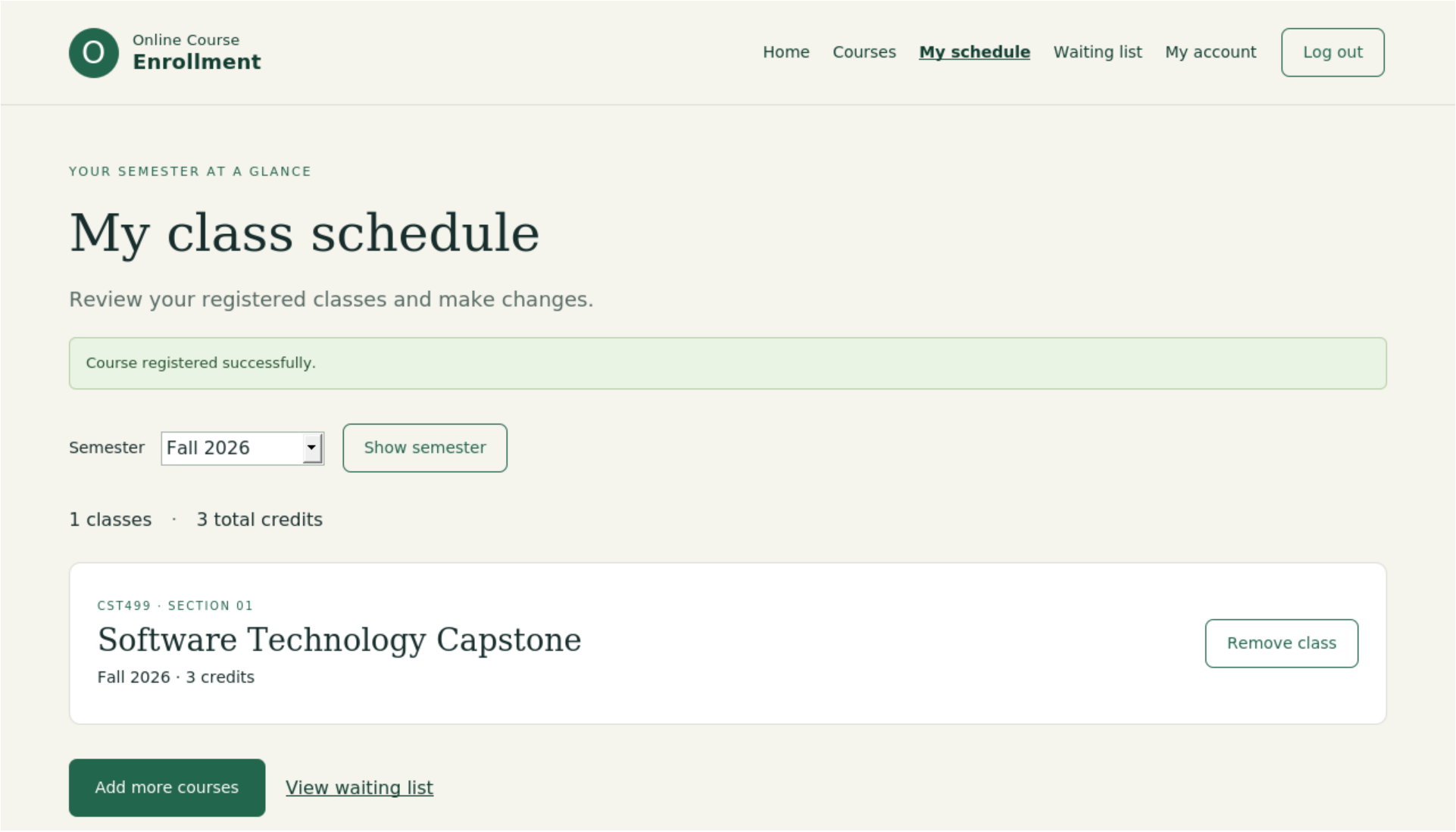


Figure 19 After Adding Another Class

After adding a class and before deletion: account_id 1 has offering_id 1 and 2.

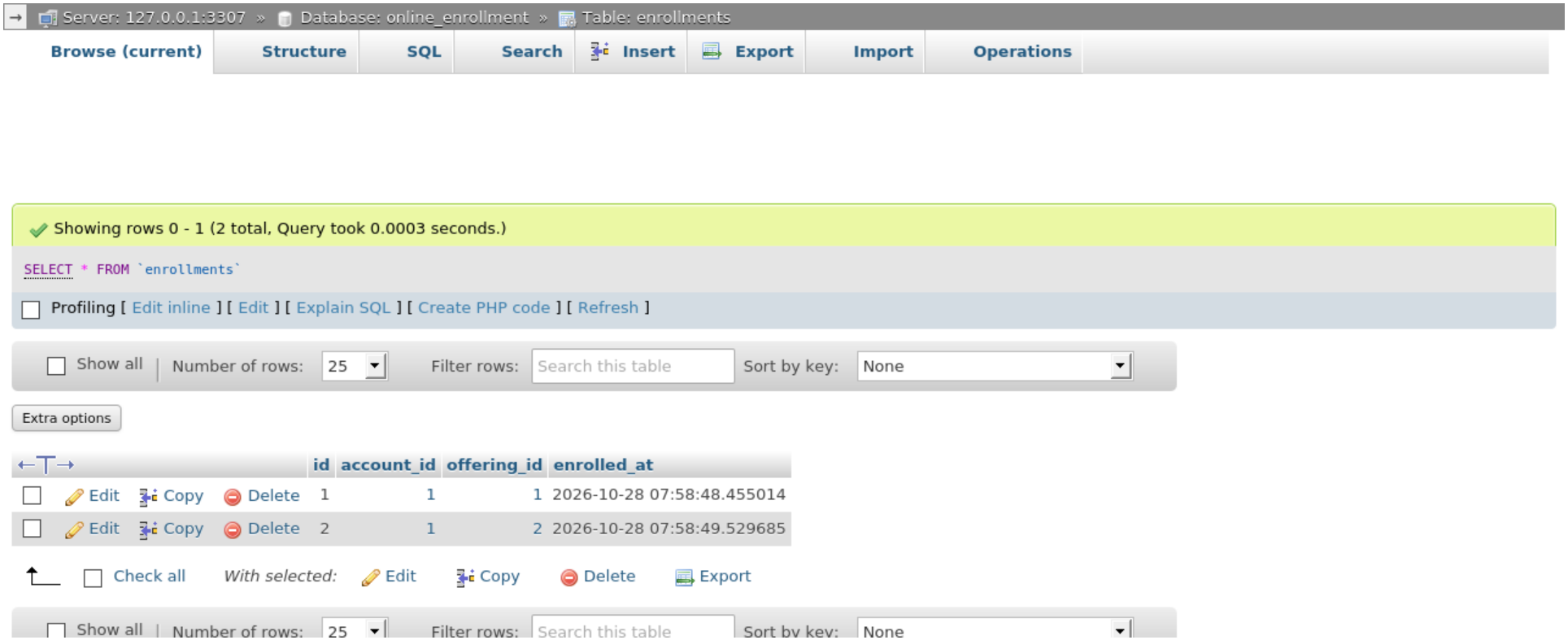


Figure 20 Schedule With Two Classes

Two registered classes total six credits; Add more courses returns to the same semester.

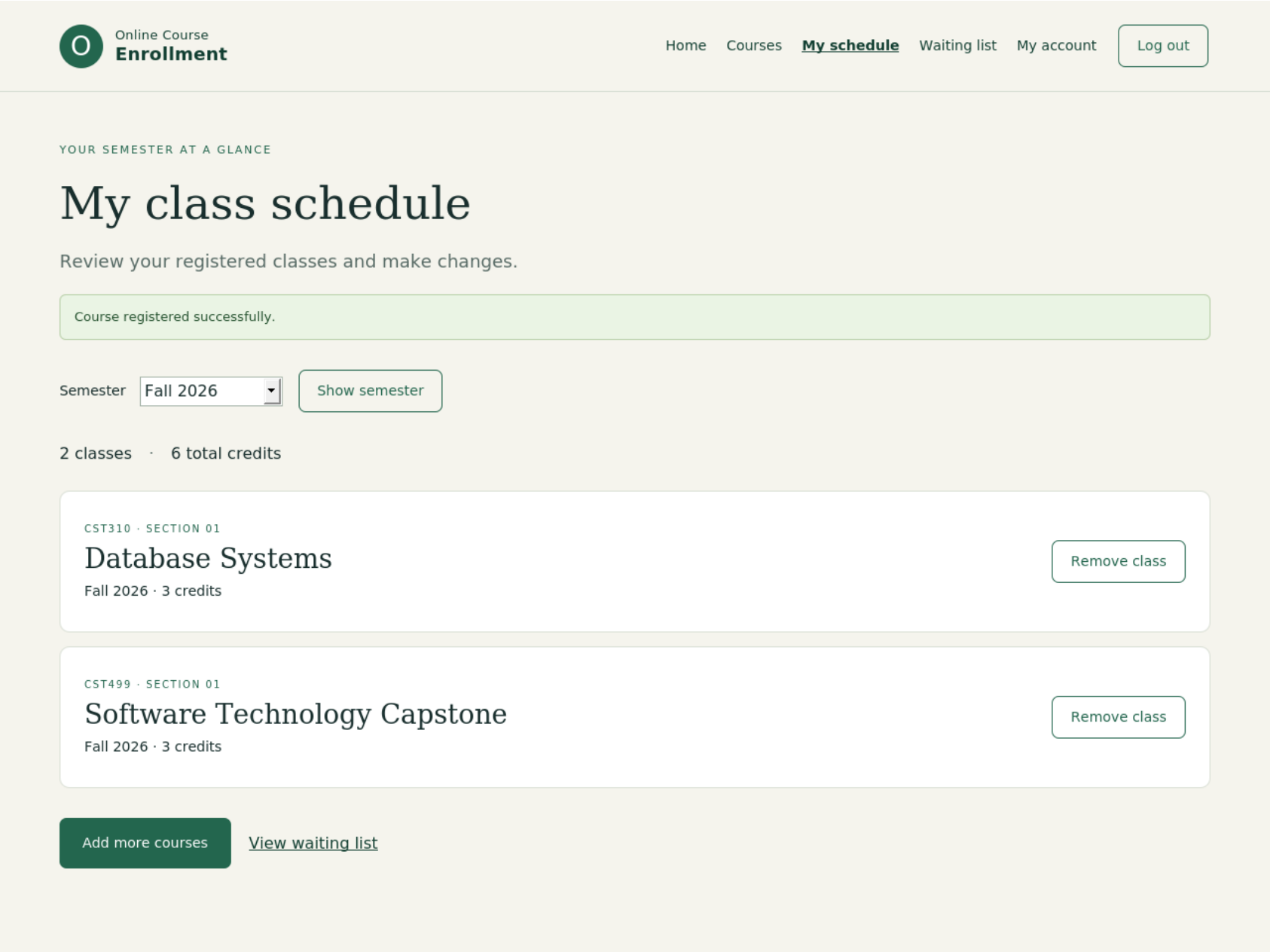


Figure 21 Class Removal Confirmation

A separate confirmation page identifies the exact class and explains the seat release.

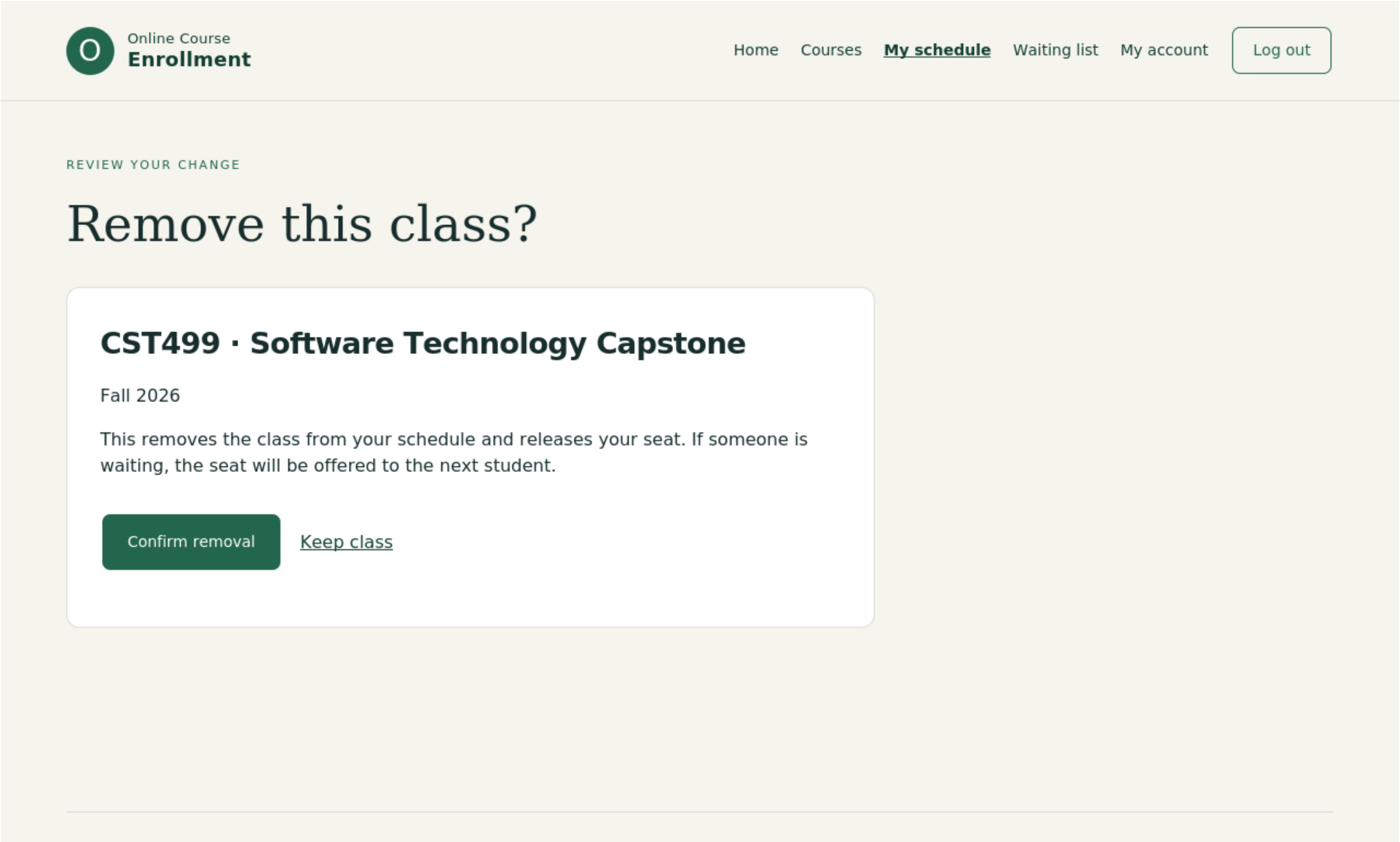


Figure 22 After Class Removal

After confirmed deletion: offering 1 is removed while offering 2 remains for the same student.

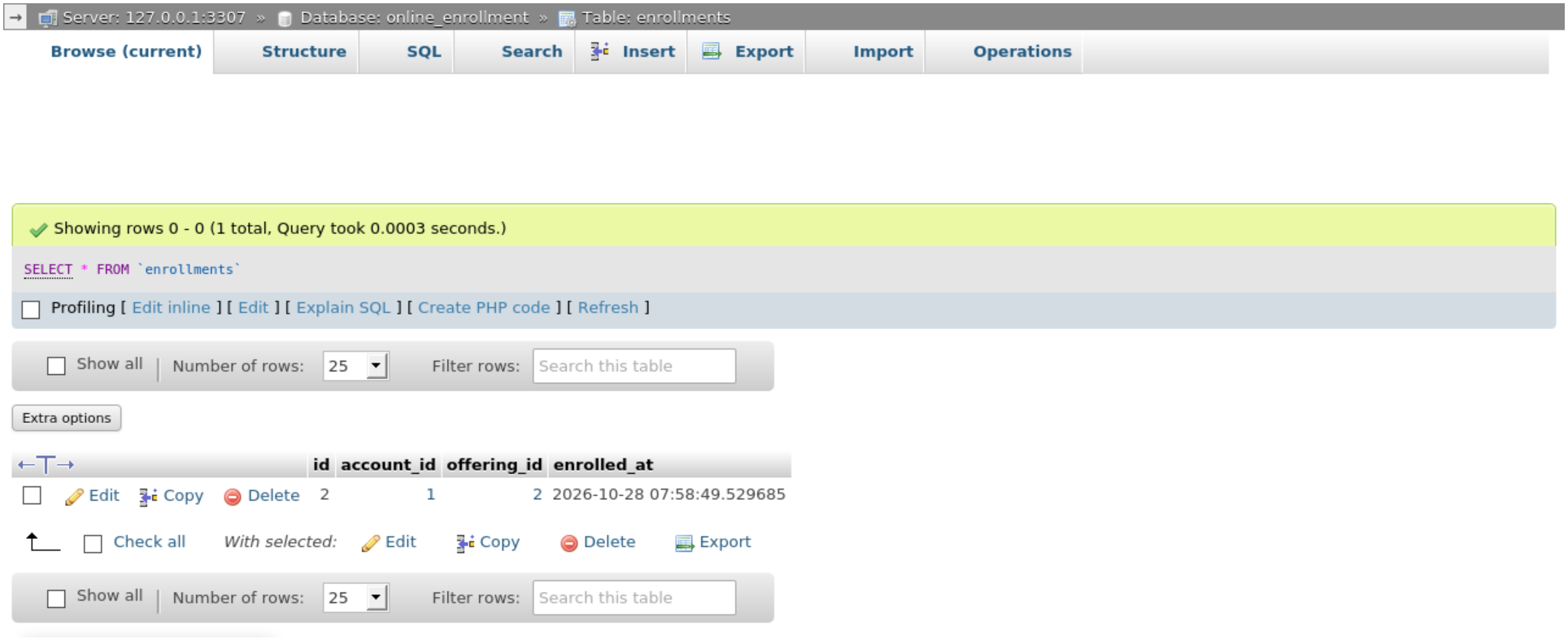


Figure 23 Updated Student Schedule

The refreshed schedule contains only Database Systems after removal.

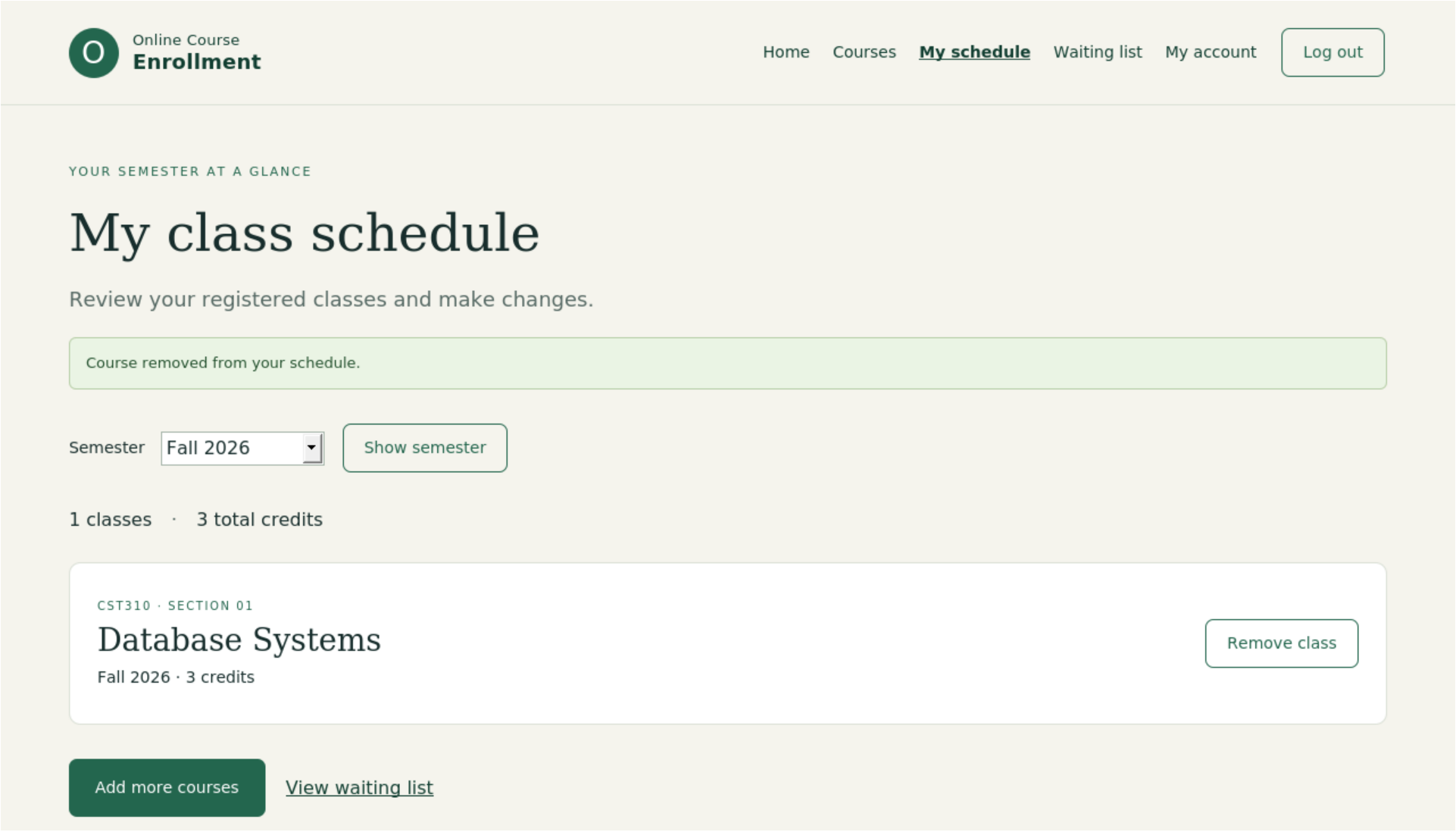


Figure 24 Empty Semester

Summer 2027 has no seeded offerings and displays the empty-term message.

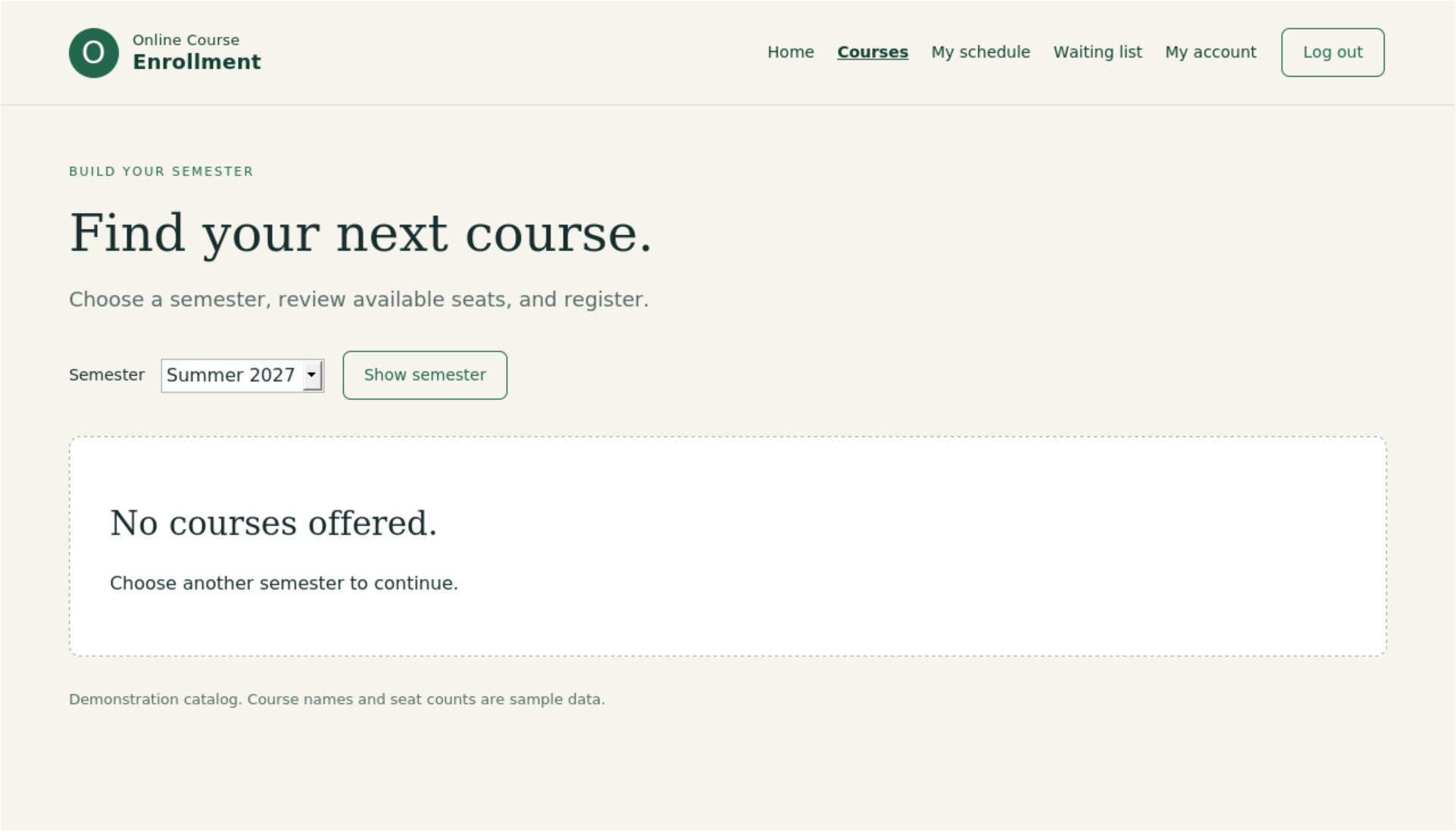


Figure 25 Waiting List Position

The waiting-list page identifies the course, semester, and queue position.

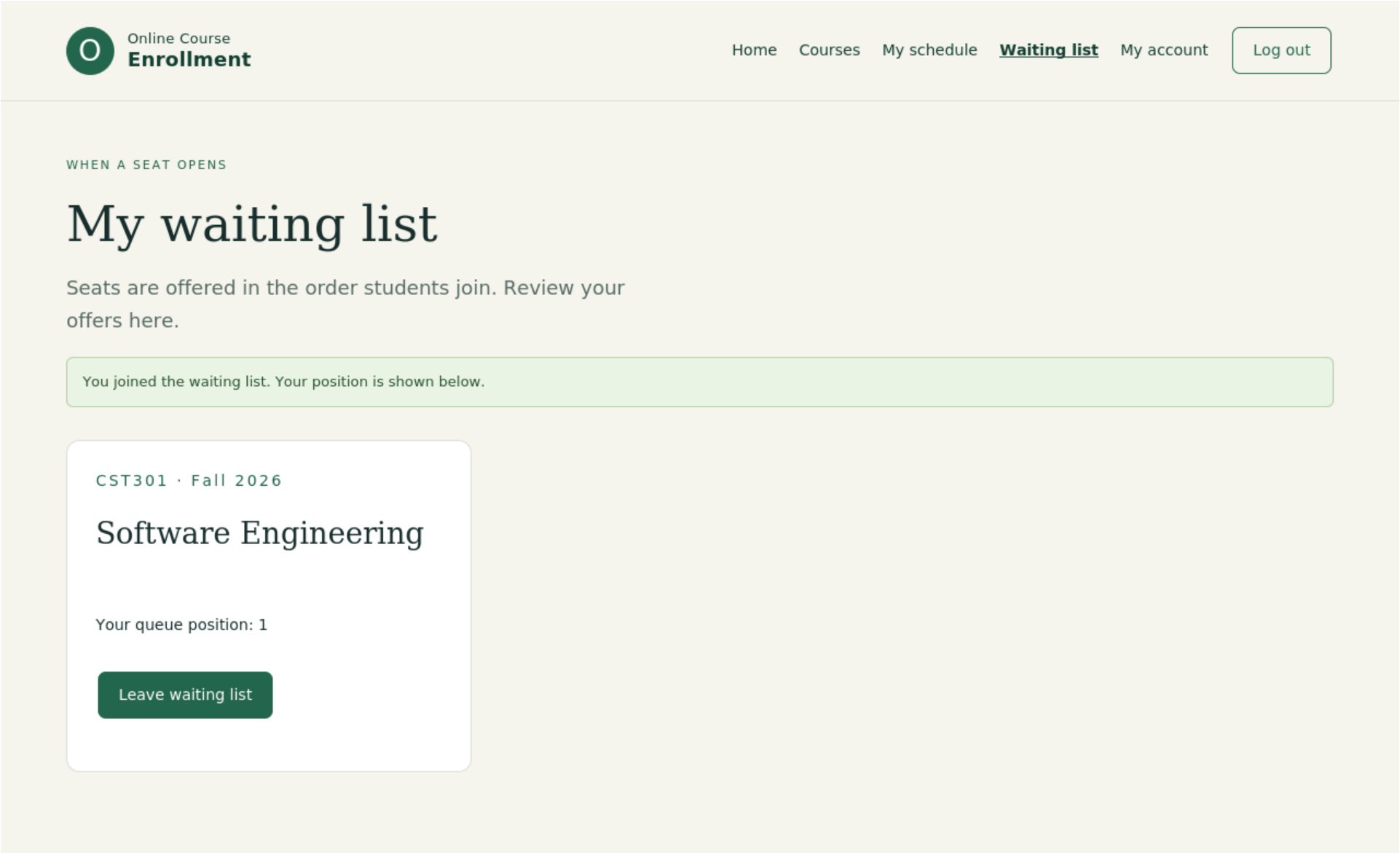


Figure 26 Reserved Seat Notice

Cancellation creates a reserved-seat notice for Alex with an acceptance link and expiration.

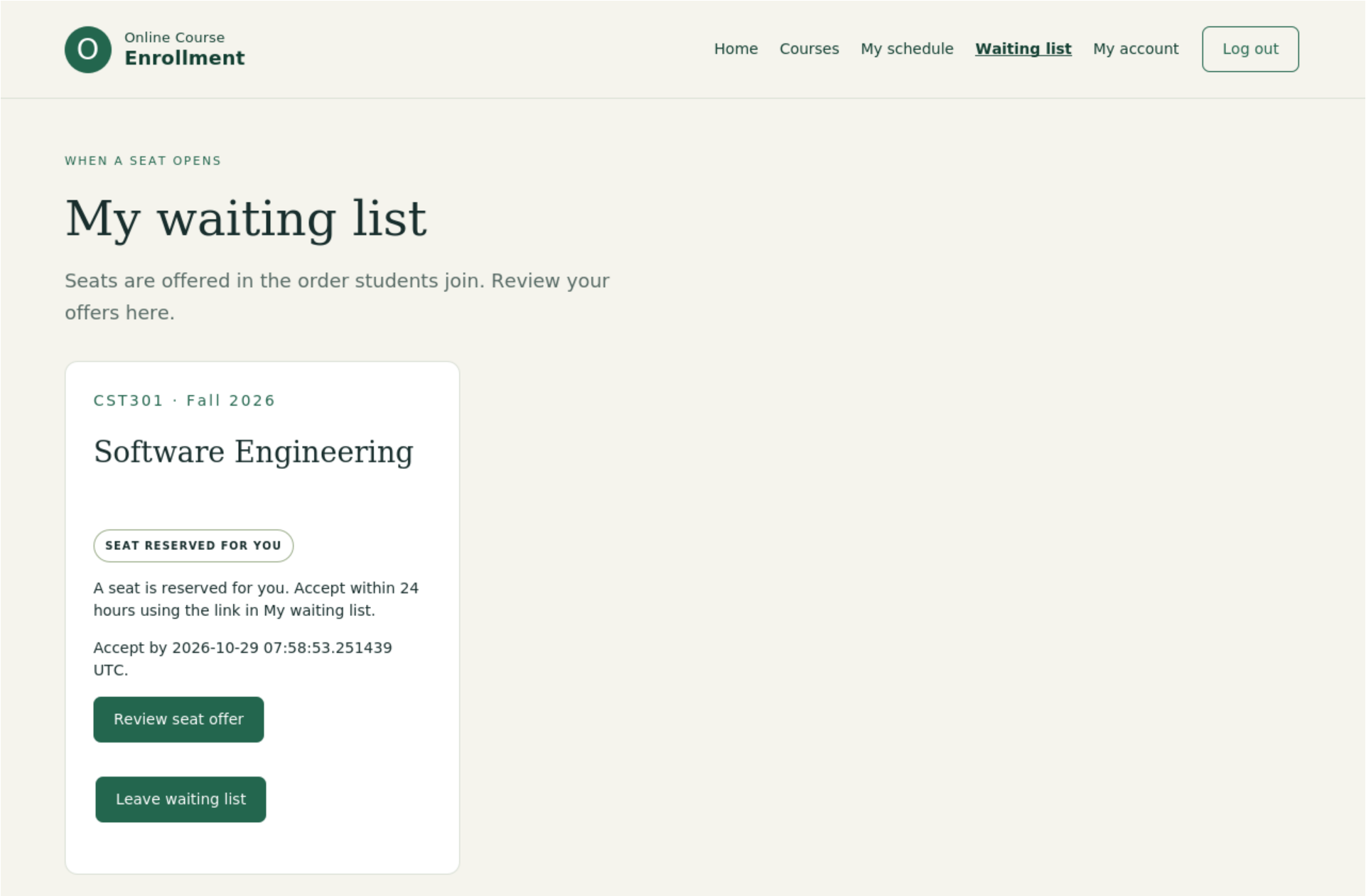


Figure 27 Seat Offer Decision

The offer page supports explicit acceptance or decline.

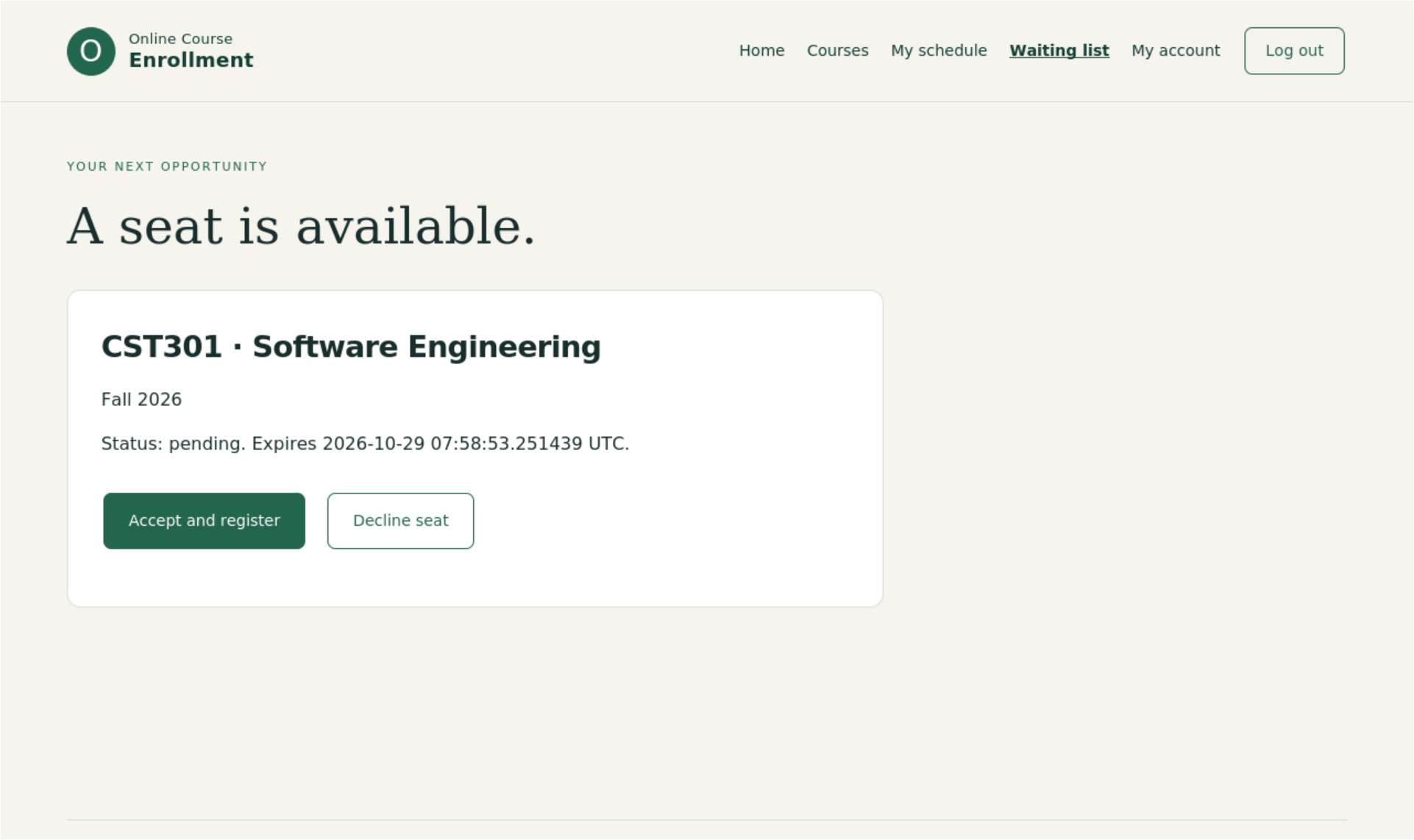


Figure 28 Before Offer Acceptance

Before acceptance: one pending reservation retains its recipient and waiting-list entry.

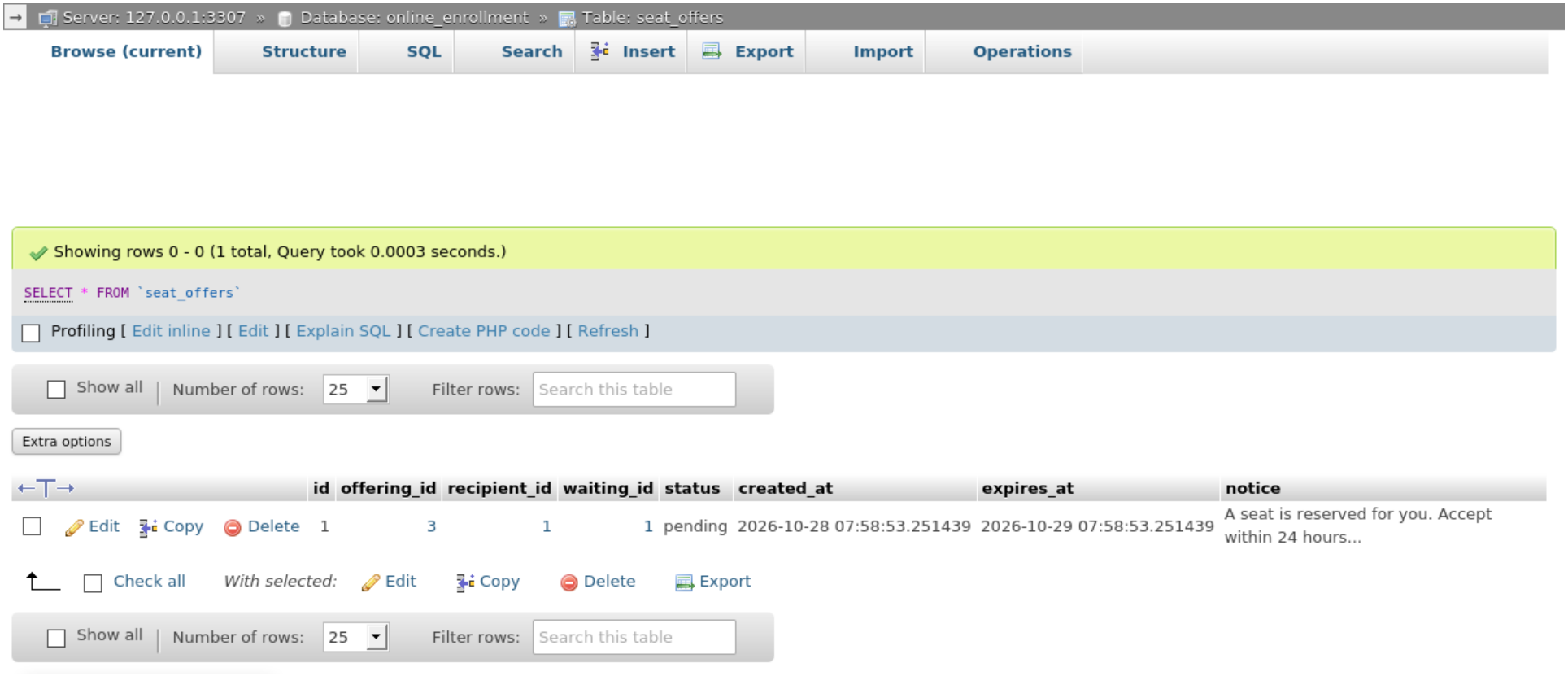


Figure 29 After Offer Acceptance

After acceptance: status is accepted and waiting_id is cleared; the recipient remains recorded.

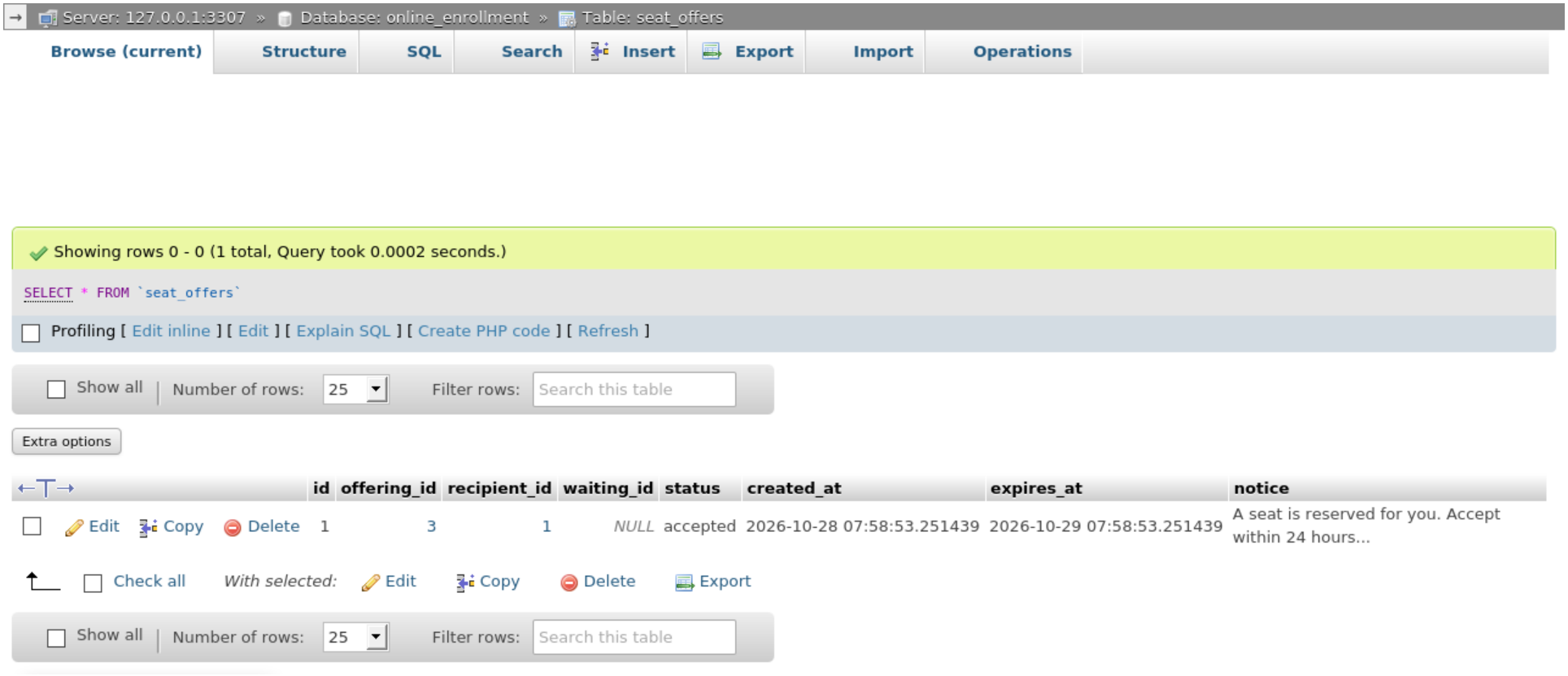


Figure 30 Profile Editing

Profile editing retains the existing account identity while allowing contact updates.

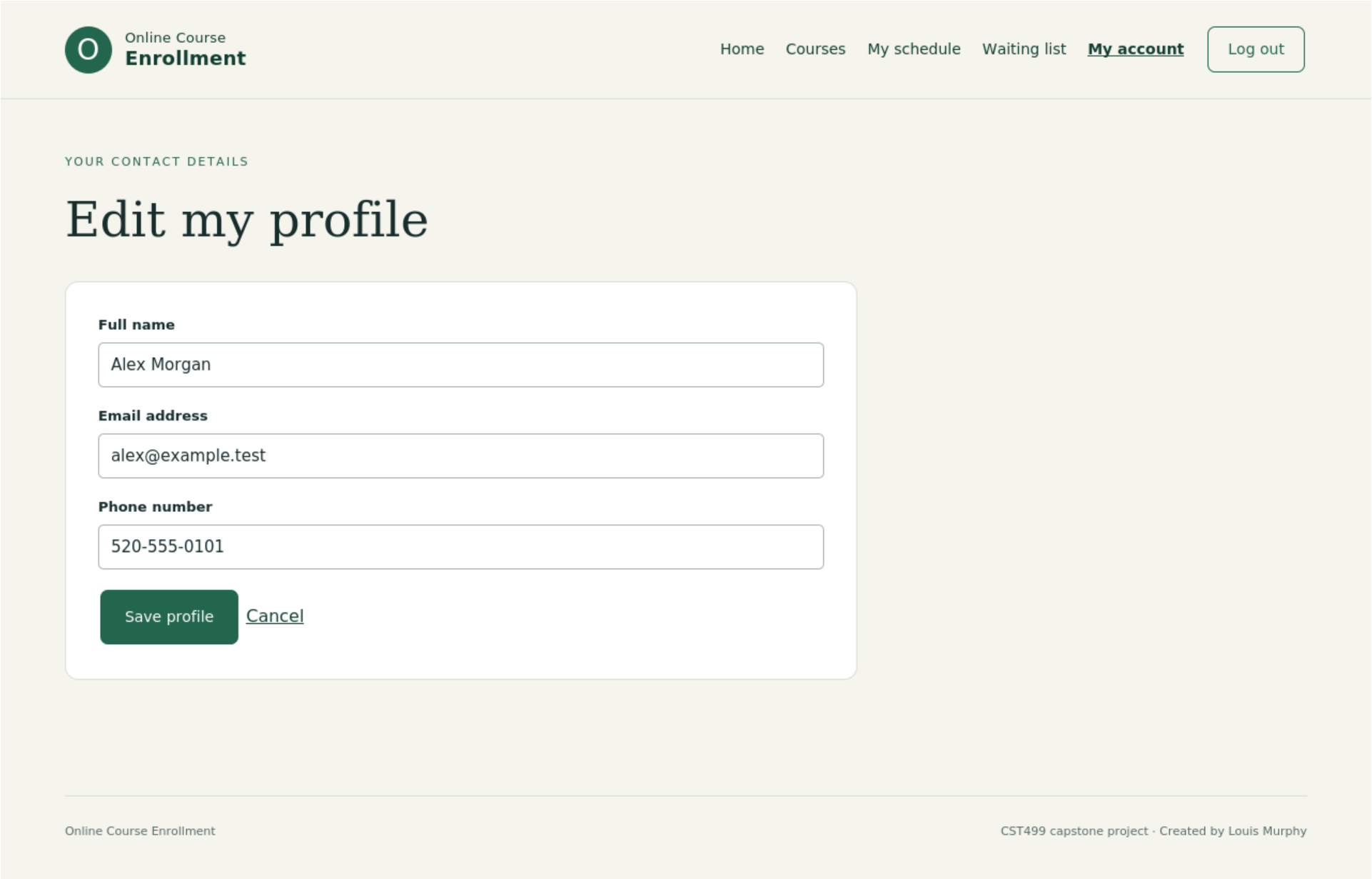


Figure 31 Transaction and Rollback Code

Actual PHP source from app/EnrollmentService.php, lines 7-31.

app/EnrollmentService.php

Actual project source · lines 7–31

```
 7
 8      private function query(string $sql, array $params = []): PDOStatement
 9      {
10          $statement = $this->db->prepare($sql);
11          $statement->execute($params);
12          return $statement;
13      }
14
15      // Every seat-changing operation obtains the same offering lock first.
16      private function transaction(int $offeringId, callable $operation): mixed
17      {
18          $this->db->beginTransaction();
19          try {
20              $offering = $this->query('SELECT * FROM offerings WHERE id = ? FOR UPDATE', [$offeringId])->fetch();
21              if (!$offering) { throw new DomainException('Course offering not found.'); }
22              $this->refreshOffers($offering);
23              $result = $operation($offering);
24              $this->db->commit();
25              return $result;
26          } catch (Throwable $error) {
27              if ($this->db->inTransaction()) { $this->db->rollBack(); }
28              throw $error;
29          }
30      }
31
```

Figure 32 Registration and Removal Code

Actual PHP source from app/EnrollmentService.php, lines 59-81.

app/EnrollmentService.php

Actual project source · lines 59–81

```
 59      public function enroll(int $accountId, int $offeringId): string
 60      {
 61          return $this->transaction($offeringId, function (array $offering) use ($accountId, $offeringId) {
 62              if ($this->query('SELECT id FROM enrollments WHERE account_id = ? AND offering_id = ?', [$accountId,
$offeringId])->fetch()) {
 63                  return 'You are already registered for this course.';
 64              }
 65              if ($this->query('SELECT id FROM waiting_list WHERE account_id = ? AND offering_id = ?', [$accountId,
$offeringId])->fetch()) {
 66                  return 'You are on the waiting list. Review any seat offer there.';
 67              }
 68              if ($this->remaining($offering) < 1) { return 'This course is full. You may join the waiting list.'; }
 69              $this->query('INSERT INTO enrollments (account_id,offering_id) VALUES (?,?)', [$accountId, $offeringId]);
 70              return 'Course registered successfully.';
 71          });
 72      }
 73
 74      public function cancel(int $accountId, int $offeringId): string
 75      {
 76          return $this->transaction($offeringId, function (array $offering) use ($accountId, $offeringId) {
 77              $deleted = $this->query('DELETE FROM enrollments WHERE account_id = ? AND offering_id = ?', [$accountId,
$offeringId])->rowCount();
 78              $this->refreshOffers($offering);
 79              return $deleted ? 'Course removed from your schedule.' : 'No active enrollment was changed.';
 80          });
 81      }
```

Figure 33 Protected Request Handler

Actual PHP source from public/enroll.php, lines 1-19.

public/enroll.php

Actual project source · lines 1–19

```
  1  <?php
  2  require_once __DIR__ . '/../app/bootstrap.php';
  3  $student = require_student();
  4  if ($_SERVER['REQUEST_METHOD'] !== 'POST') { http_response_code(405); header('Allow: POST'); exit('Use a course action
form.'); }
  5  if (!valid_csrf()) { http_response_code(403); exit('The request could not be verified.'); }
  6  try {
  7      $service = new EnrollmentService(db());
  8      $offering = positive_id(post('offering_id'));
  9      $semester = positive_id(post('semester'));
 10      $action = post('action');
 11      $_SESSION['flash'] = match ($action) {
 12          'enroll' => $service->enroll($student,$offering),
 13          'join' => $service->join($student,$offering),
 14          'leave' => $service->leave($student,$offering),
 15          default => throw new DomainException('Unknown course action.'),
 16      };
 17      redirect($action === 'enroll' ? 'schedule.php?semester=' . $semester : 'waiting.php');
 18  } catch (DomainException $ex) { http_response_code(400); exit(e($ex->getMessage())); }
 19  catch (Throwable $ex) { error_log('Course action failed.'); http_response_code(503); exit('The change was not saved.
Please try again.'); }
```

Figure 34 Catalog and Schedule Queries

Actual PHP source from app/EnrollmentService.php, lines 137-151.

app/EnrollmentService.php

Actual project source · lines 137–151

```
137     public function catalog(int $accountId, int $semesterId): array
138     {
139         return $this->query("SELECT o.*,c.code,c.title,c.credits,s.year,s.term,
140           o.capacity - (SELECT COUNT(*) FROM enrollments e WHERE e.offering_id=o.id)
141           - (SELECT COUNT(*) FROM seat_offers f WHERE f.offering_id=o.id AND f.status='pending') AS remaining,
142           EXISTS(SELECT 1 FROM enrollments e WHERE e.offering_id=o.id AND e.account_id=?) AS enrolled,
143           EXISTS(SELECT 1 FROM waiting_list w WHERE w.offering_id=o.id AND w.account_id=?) AS waiting
144           FROM offerings o JOIN courses c ON c.id=o.course_id JOIN semesters s ON s.id=o.semester_id
145           WHERE o.semester_id=? ORDER BY c.code", [$accountId,$accountId,$semesterId])->fetchAll();
146     }
147
148     public function schedule(int $accountId, int $semesterId): array
149     {
150         return $this->query('SELECT e.*,c.code,c.title,c.credits,o.section,s.term,s.year FROM enrollments e JOIN
offerings o ON o.id=e.offering_id JOIN courses c ON c.id=o.course_id JOIN semesters s ON s.id=o.semester_id WHERE
e.account_id=? AND o.semester_id=? ORDER BY c.code', [$accountId,$semesterId])->fetchAll();
151     }
```